REQUI TRADING · ENGINEERING

Deterministic Strategy Engine & Auto-Universe

DevOps Deployment Guide v2.5 — amends v2.2 (Autonomous Execution) and v2.4 (Trailing & Exit Engine). One hundred strategies embedded as versioned data objects. The bot builds its own watchlist: seed universe → deterministic filter → rank → strategy assignment → monitors → ticket gate → broker — with the trailing engine retaining exclusive ownership of every protective exit.

100 STRATEGIES EMBEDDED | AUTO-UNIVERSE ENGINE | ENGINE HEARTBEAT 60s | GLOBAL RISK GATES §4 | TRAILING COEXISTENCE

Audience: DevOps / Backend Engineering Strategy Specification v1.0 implementation August 2026 · CONFIDENTIAL

Contents

1 What changed — executive summary for DevOps

The request. Until this release, autonomous trading required a human to type symbols into the data feed and start monitors by hand. Strategy Specification v1.0 changes the operating model: **the bot lists the stocks itself**, one hundred strategies are **embedded into the deterministic engine as versioned data objects**, and the whole layer **coexists with the trailing-stop engine** shipped in v2.4 without touching its exit authority.

What ships in v2.5:

Capability	What it does	Where
Strategy registry (100 strategies)	Every REQ-001…REQ-100 from the specification embedded as a Universal Strategy Object (§4 of this guide). Status lifecycle enforced: only `APPROVED` + executable strategies may trade. Config hash per strategy for audit (§10 spec).	`api/engine/strategies/`
Auto-universe engine	The bot builds the watchlist: ~65-symbol liquid seed universe → deterministic filter (price ≥ $5 · $20M session dollar volume · rvol ≥ 1.2 · fresh data) → rank → assign an APPROVED strategy by tape shape → start monitors automatically. Replaces manual symbol tracking for opted-in users.	`api/engine/universe.ts`
Engine heartbeat	A 60-second market-hours loop that refreshes all feeds, evaluates *every* active monitor (previously monitors only moved when a human clicked “evaluate”), then runs auto-universe cycles. **This fixes the frozen-screen behavior reported in UAT.**	`ensureUniverseLoop()` in `boot.ts`
Global risk gates (spec §4)	Portfolio-heat gate (10% cap) and pairwise-correlation gate (0.70) now sit in the live proposal path before any ticket is staged. Unverifiable data → deterministic REJECT (spec §5 never-estimate rule).	`api/engine/monitor.ts`, `risk.ts`

First-release executable strategies	REQ-001 (Opening Range Breakout Long) and REQ-003 (VWAP Reclaim Long) are APPROVED and executable on current primitives, with a new deterministic `orb_breakout` trigger. The remaining eight first-release strategies are registered with their exact blockers documented.	`triggers.ts`, `config.ts`, `library.ts`
Operator UI	UniversePanel: auto-universe toggle, force-scan, last-cycle candidate table with per-symbol deterministic decisions, and the full 100-strategy registry with status badges and operator-kill controls.	`src/components/UniversePanel.tsx`

HARD BOUNDARIES PRESERVED

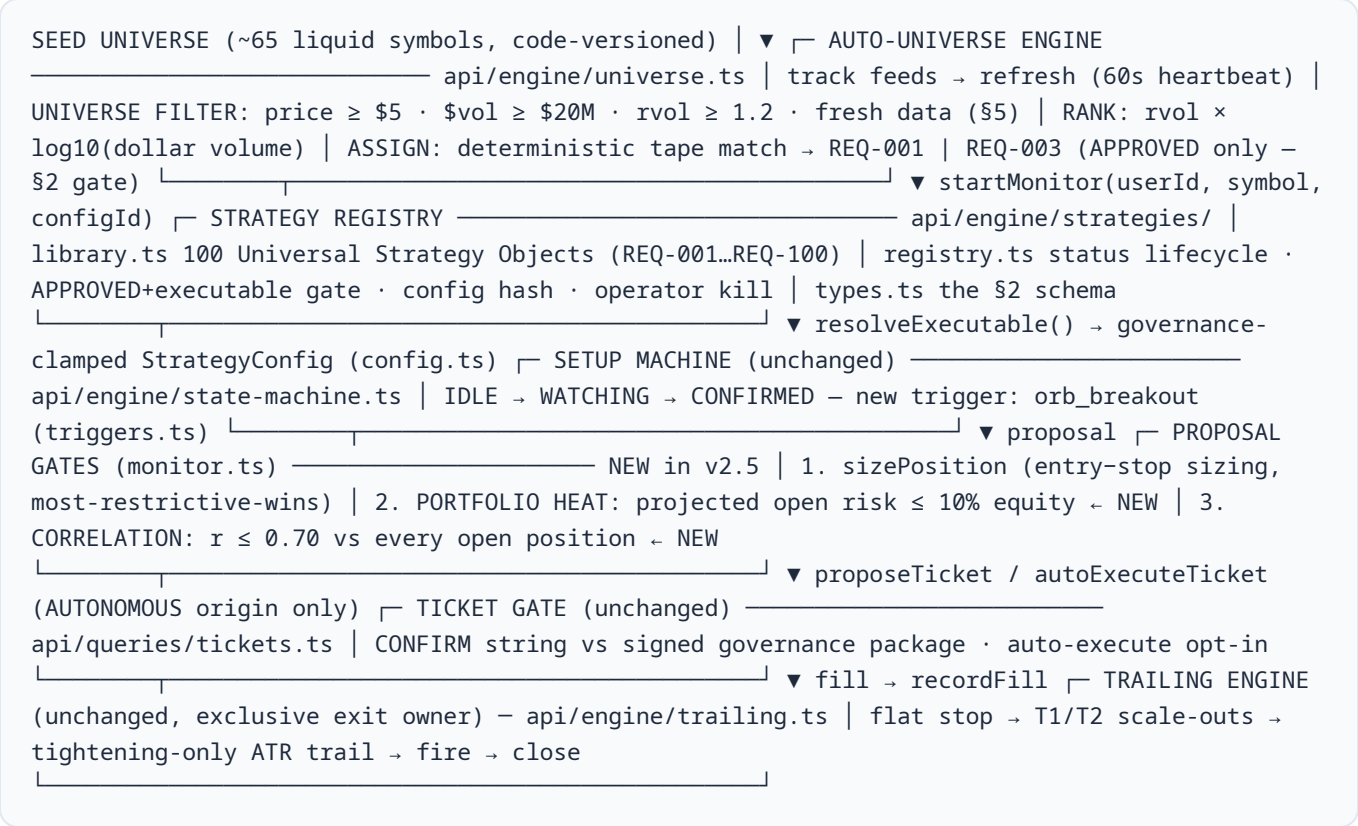
The AI layer may scan, rank and explain. It may **never** override a parameter, stop, size or gate. Strategies seed entries through the existing ticket pipeline; the **trailing engine keeps exclusive ownership of every protective exit**. Auto-execute remains scoped to AUTONOMOUS-origin tickets only, default OFF, fail-closed. Governance package verification remains load-bearing at boot and on every load.

SCHEMA MIGRATION — TWO COLUMNS, GUARDED

`users.autoUniverse TINYINT(1) NOT NULL DEFAULT 0` and `users.autoUniverseMax INT NOT NULL DEFAULT 5`. Both ship as guarded ALTERs in `ensure-schema.ts` — idempotent, safe to run against the live database, no downtime, no data rewrite.

2 Architecture map

The v2.5 layer slots into the existing engine without modifying its execution contract. New modules are shaded; everything downstream of the ticket gate is unchanged from v2.2/v2.4.



Separation of powers (spec §15) preserved: the strategy layer decides *what* to enter and the risk layer decides *whether*; the ticket layer decides *if a human confirmed*; the trailing layer decides *when to exit*. No layer can do another layer's job.

3 Deployment procedure

3.1 Files added

File	Lines	Purpose
`api/engine/strategies/types.ts`	97	Universal Strategy Object schema + status/decision enums
`api/engine/strategies/library.ts`	366	All 100 REQ strategy objects (10 first-release full, 90 compact-complete)
`api/engine/strategies/registry.ts`	122	Lifecycle gate, config hash, operator kill, eligibility resolution
`api/engine/universe.ts`	282	Auto-universe engine + 60s engine heartbeat
`src/components/UniversePanel.tsx`	208	Operator UI: auto-universe + registry

3.2 Files modified

File	Change
`api/engine/triggers.ts`	New `orb_breakout` trigger (opening-range high/low from first N RTH bars, rvol evidence, chase limit)
`api/engine/config.ts`	Two new governance-clamped configs: `req_001_orb_long`, `req_003_vwap_reclaim`
`api/engine/risk.ts`	`GLOBAL_RISK` constants (§4) + `portfolioHeatPct()` + `returnCorrelation()`
`api/engine/monitor.ts`	Portfolio-heat and correlation gates inserted in the proposal path (deterministic REJECT with reason)
`api/engine/portfolio.ts`	Auto-universe preference queries, `listAutoUniverseUsers()`, `openRiskDollars()`
`db/schema.ts`	Users columns `autoUniverse`, `autoUniverseMax`
`api/lib/ensure-schema.ts`	Two guarded ALTERs appended to `COLUMN_STATEMENTS`
`api/engine-router.ts`	Eight new endpoints (§11 of this guide)
`api/boot.ts`	`ensureUniverseLoop()` added to the production boot sequence
`src/pages/app/Autonomous.tsx`	`<UniversePanel />` mounted after EnginePanel

3.3 Rollout steps

1. **Deploy the build.** Standard pipeline: `npm run build` → `dist/boot.js` + `dist/public`. No new environment variables, no new services, no new ports.
2. **Restart.** On boot: `ensureSchema()` applies the two guarded ALTERs (idempotent), `ensureGovernanceReady()` re-verifies the signed package, then `ensureUniverseLoop()` arms the 60s heartbeat. Expected log: `[db] schema ensured` then `Server running`.
3. **Smoke-verify.** `GET /api/trpc/engine.strategyStats` must return 401 unauthenticated (not 404). Signed-in: registry returns 100 rows, 2 eligible.
4. **Enable per user.** Auto-universe is OFF by default (fail-closed). Users opt in from the Autonomous page; monitor cap defaults to 5, max 20.
5. **Rollback.** Version snapshot `b6854db` precedes any later change; rollback restores the full project state. The two new user columns are harmless to older code (unused, defaulted).

OPERATIONAL NOTES

Seed tracking is batched (5 symbols / 300ms) to respect the IBKR gateway. The heartbeat refreshes feeds itself, so the universe cycle always evaluates fresh data. Feeds are a shared global map — N users with auto-universe do not multiply gateway load. During closed market hours the heartbeat stands down entirely (no orders, no evaluations, no cycles).

4 The Universal Strategy Object (spec §2)

Every strategy — intraday equity, options structure, pairs trade, factor portfolio — is expressed as one schema. The object is **data, not code**: the deterministic pipeline interprets it, and its SHA-256 config hash is recorded wherever the strategy is used, so a parameter change is always detectable (spec §10/§11).

Field	Meaning
`strategy_id / name / version`	REQ-### identifier, display name, semver. Trades record the version used.
`status`	DRAFT · BACKTEST_REQUIRED · PAPER_TRADING · CANARY · APPROVED · DISABLED. Only APPROVED is eligible for autonomous trading.
`asset_class / direction / timeframe`	EQUITY/ETF/OPTIONS/FUTURES · LONG/SHORT · INTRADAY/SWING/EVENT/POSITION.
`universe`	minPrice ($5), minDollarVolume, maxSpreadPct, session — the REQ-001 SCAN contract every strategy inherits.
`regime / required_data / indicators`	Allowed market regimes; the data the strategy may not run without (§5); indicator dependencies.
`scan_rules / signal_rules / confirmation_rules`	Auditable text of each deterministic gate, in pipeline order.
`entry / position_sizing / initial_stop`	Entry contract; risk_pct 0.50 with hard_cap_pct 1.00 (§4); stop construction.
`profit_targets / trailing_stop / time_exit`	T1/T2 in R-multiples; the hand-off sentence to the trailing engine; session cutoff rule.
`liquidity_rules / portfolio_rules`	liquidity score ≥ 0.50, VPIN ≤ 0.70; portfolio heat ≤ 10%, pairwise correlation ≤ 0.70 (§4 globals).
`event_rules / no_trade_rules`	Feed-down → HALT; stale/missing → REJECT, never estimate. The deterministic refusal list.
`exit_rules`	Priority order per spec §7 — kill switch first, trailing exit last.
`validation`	The exact evidence chain standing between this strategy and APPROVED.
`executable`	null = metadata-only (not activatable). Present = mapping onto engine primitives (config id + triggers + honest approximation notes).

WHY DATA OBJECTS, NOT STRATEGY CODE

A strategy as an object is hashable, diffable, and clampable. The engine reads it; governance clamps it; the audit ledger hashes it. Nobody hot-patches a number in production — a change is a release with a new version and a new hash, which is exactly what spec §11 demands.

5 Registry, lifecycle & the APPROVED gate

The registry is the engine's read-path over the embedded library. It enforces the lifecycle from spec §14 and the eligibility rule from spec §2 — in code, not in convention.

```
DRAFT → BACKTEST_REQUIRED → BACKTEST_PASS → WALK_FORWARD_PASS → OUT_OF_SAMPLE_PASS →
PAPER_TRADING → CANARY → APPROVED (any failure demotes to BACKTEST_REQUIRED)
```

5.1 Eligibility is two conditions, always

```
eligible(strategy) ⇔ status === "APPROVED" AND executable ≠ null
```

`resolveExecutable(id)` is the single gate through which the universe builder and every monitor start must pass. It throws `STRATEGY_NOT_ELIGIBLE` with the exact reason otherwise. There is no second path.

5.2 Runtime can only demote

The only status mutation the runtime permits is the **operator kill**: APPROVED → DISABLED, effective immediately for new entries, with open positions unaffected (their exits remain with the trailing engine). `enableStrategy()` merely restores the library status — it can never promote beyond what the versioned code ships. Promotion requires the §14 evidence chain plus a versioned release, which is the only way a hash-changing edit should ever reach production.

5.3 Config hash

Each strategy object hashes to a 16-hex-char SHA-256 fragment shown in the UI registry table and intended for the audit ledger wherever the strategy is referenced. Same library → same hash, every boot, every environment — a cheap integrity tripwire against drift between environments.

6 Auto-universe engine — the bot lists the stocks

This module replaces the manual “type a symbol, click track, click monitor” workflow for opted-in users. Every step is deterministic; every refusal carries a machine-readable reason.

6.1 The cycle

1. **Track the seed universe** (~65 liquid symbols: index/sector ETFs plus mega-cap names; code-versioned, batched 5-at-a-time against the gateway).
2. **Filter** — a symbol must pass all of: last price ≥ $5; session dollar volume ≥ $20M; rvol ≥ 1.2 against the prior session's same-window volume; feed fresh (< 180s) and error-free (§5: stale or missing data is a REJECT, never an estimate).
3. **Rank** — `score = rvol × log10(session dollar volume + 1)`.
4. **Assign** — deterministic tape match against APPROVED registry strategies only: price above the 15-minute opening-range high with rvol ≥ 1.5 → **REQ-001**; traded below VWAP earlier and now above → **REQ-003**. No match → **WAIT** with the reason recorded.
5. **Start monitors** — top-ranked assigned candidates, up to the user's cap (default 5, hard max 20). Manual monitors are never duplicated or overridden; per-day OCO remains enforced by the state machine.

6.2 Deterministic decision vocabulary

Decision	When
MONITOR	Passed filter, matched an APPROVED strategy, open monitor slot — monitor started.
WAIT	No APPROVED strategy pattern matches the tape · monitor cap reached · symbol already monitored.
REJECT	Data failure (stale/errored feed) or monitor start refused by a downstream gate.
HALT	Market closed or feed service down — the heartbeat stands down; no new entries.

6.3 What it deliberately does not do

- It never sizes, never stages tickets, never touches stops — that is the monitor → risk gates → ticket → trailing chain's job.
- It never activates a non-APPROVED strategy. Eight of the ten first-release strategies remain gated behind their documented blockers (§10).
- It never estimates a missing value. A symbol without fresh, complete bars is skipped with a reason, per spec §5.

> **USER-FACING CONTRACT**
>
> Auto-universe is opt-in per user (`autoUniverse` , default OFF, fail-closed on read error). The operator sees every cycle in the UI: tracked/filtered counts, the ranked candidate table, per-symbol decisions and reasons, and which monitors were started. Nothing trades in the dark.

7 The engine heartbeat (fixes frozen monitors)

Pre-v2.5, `evaluateAll()` existed but had no caller: monitors advanced only when a human clicked “evaluate” in the UI. With the browser closed or the market quiet, the screen appeared stuck — the state machines were simply never ticked. v2.5 closes the gap with a proper engine heartbeat.

```
ensureUniverseLoop() — every 60s, market hours only: 1. marketDataService.refreshAll() → every tracked feed refreshed 2. evaluateAll() → EVERY active monitor's state machine ticks 3. for each auto-universe user → ensureSeedUniverse() → runUniverseCycle()
```

Properties that matter operationally:

- **Market-hours gated** (`isMarketHours()` , ET, Mon–Fri 09:30–16:00). Overnight and weekends the engine is fully stood down.
- **Re-entrancy guarded** — a slow gateway cannot stack overlapping ticks.
- **Unref'd timer** — the loop never keeps the process alive on its own.
- **Fail-closed roster** — if the database cannot be read, the auto-universe user list is empty and no cycles run; manual monitors still tick via step 2.

RESULT

Monitors now advance continuously through IDLE → WATCHING → CONFIRMED on live data without any human in the loop; proposals surface as tickets through the same confirmation (or auto-execute) gate as before. The “stuck screen” from UAT was correct behavior for a closed market plus a missing heartbeat — both causes are now addressed.

8 Global risk gates (spec §4) in the proposal path

The specification's global risk layer is now constants in `risk.ts` and two enforced gates in `monitor.ts`, evaluated on every CONFIRMED proposal **before** any ticket is staged. Both gates fail closed with a deterministic reason string that lands in the monitor detail and the activity stream.

8.1 Constants

Constant	Value	Source
`defaultPositionRiskPct`	0.50%	§4 default per-position risk
`hardPositionRiskCapPct`	1.00%	§4 absolute cap (governance clamps configs above it)
`portfolioHeatMaxPct`	10%	§4 total open-risk ceiling
`maxPairwiseCorrelation`	0.70	§4 correlation gate
`liquidityScoreMin` / `vpinMax`	0.50 / 0.70	§4 (declared; enforced where data exists)
`drawdownBrakePct`	10%	§4 account drawdown brake

8.2 Gate 10 — portfolio heat

```
heat = Σ open-risk dollars / equity × 100 open-risk per position = qty × (entry − protective stop) [floored at 0] unprotected position = qty × entry [FULL notional — most conservative] projected = heat + new position's dollar risk / equity × 100 projected > 10% → REJECT: "proposal blocked by PORTFOLIO HEAT gate — projected X% > 10% cap"
```

8.3 Gate 11 — pairwise correlation

```
r = Pearson correlation of 1-min close-to-close returns, candidate vs each open position, trailing 30-bar aligned window r > 0.70 → REJECT: "r=0.74 vs open position XYZ exceeds 0.70" r unverifiable → REJECT: "cannot verify vs open position XYZ (no feed data; §5 never-estimate rule)" zero-variance side → r = 0 (a series that never moves cannot co-move)
```

HONEST LIMITATION, DOCUMENTED IN CODE

The correlation gate consumes tracked 1-minute feeds. Open positions are normally tracked (the trailing engine marks them), so verification almost always succeeds; when it cannot, the gate refuses rather than estimates. Liquidity score and VPIN are declared in the registry and strategy objects but are **not** enforced numerically until quote/toxicity data adapters land — that gap is stated in the strategy objects' `validation` fields rather than papered over.

9 Coexistence contract with the trailing engine

The trailing engine (v2.4) owns every protective exit. The strategy engine (v2.5) owns every entry decision. The two meet at exactly one interface: **the ticket**.

```
strategy → SetupMachine proposal → risk gates → ticket (entry, stop, T1, T2 seeded onto the ticket) → fill → recordFill() → position row carries stopPrice / t1Price / t2Price → trailing.ts ratchets tightening-only, scales out, fires the exit
```

9.1 The contract in four sentences

1. Strategies **seed** protective levels (initial stop, T1, T2) onto tickets — they never move them afterward.
2. The trailing engine consumes those levels after the fill and owns the exit lifecycle: flat stop → T1 scale-out → breakeven ratchet → ATR trail → fire.
3. Spec §7 exit priority is preserved by construction: kill switch > broker fault > max-loss > thesis > regime > protective stop > time stop > target > **trailing exit last** — the trailing module is the terminal authority.
4. Neither module imports the other's internals across the contract; the strategy layer cannot widen a stop, and the trailing layer cannot initiate an entry.

9.2 Where each first-release strategy's exits live

Strategy	Stop seed (config.ts)	Targets	Exit owner
REQ-001 ORB Long	swing-low structure stop − 0.5×ATR (proxy for MIN(ORL, structure, ATR))	T1 +1.5R · T2 +2.5R	trailing engine after T1
REQ-003 VWAP Reclaim	below reclaim swing low − 0.5×ATR	T1 +1.5R · T2 +2.5R	trailing engine after T1

> **WHY THIS MATTERS TO DEVOPS**
>
> You can deploy, disable, or version strategies without any regression risk to exit behavior — the trailing engine's 5-second protective cycle is untouched by this release. The v2.4 guide's operational runbook remains fully valid.

10 First-release activation matrix (spec §13)

The specification names ten strategies for the first DevOps release. Activation honesty: only those whose executable mapping runs on **current** primitives are APPROVED today. The rest are registered in the library with their exact blocker in the `validation` field — visible in the UI, hashable, and impossible to activate by accident.

ID	Strategy	Status	Executable	Blocker / mapping
REQ-001	Opening Range Breakout Long	APPROVED	✓	`orb_breakout` trigger + `req_001_orb_long` config
REQ-002	Opening Range Breakdown Short	BACKTEST_REQUIRED	—	Short-selling execution path (tickets/portfolio)
REQ-003	VWAP Reclaim Long	APPROVED	✓	`vwap_reclaim` trigger + `req_003_vwap_reclaim` config
REQ-004	VWAP Rejection Short	BACKTEST_REQUIRED	—	Short-selling execution path
REQ-024	Volume Profile VAH Breakout	BACKTEST_REQUIRED	—	OBI + GEX data adapters (volume profile itself computable from 1m bars)
REQ-034	20-Day Donchian Breakout Long	BACKTEST_REQUIRED	—	Multi-session channel context in SetupMachine
REQ-036	Turtle Trend Following Long	BACKTEST_REQUIRED	—	Daily context + position-timeframe holding
REQ-064	PEAD Long	BACKTEST_REQUIRED	—	Earnings calendar + SUE data adapter
REQ-065	PEAD Short	BACKTEST_REQUIRED	—	Short path + earnings data adapter
REQ-091	Pairs Mean Reversion	BACKTEST_REQUIRED	—	Two-leg execution + cointegration harness

10.1 Promotion path for the remaining eight (ordered by effort)

1. **REQ-034 / REQ-036** — extend the SetupMachine context from (today, priorDay) to an N-session daily level series. Feeds already retain 2 days of 1-minute bars; `fetchMinuteBars` supports a 1-month window, so 20-session channels are a data-grouping change, not a new vendor.
2. **REQ-002 / REQ-004** — add SELL-SHORT order construction to the ticket layer and short positions to the portfolio ledger (mirrors the BUY path; the risk engine's entry−stop math already generalizes).
3. **REQ-064 / REQ-065** — earnings calendar + SUE adapter behind the existing governance approved-sources manifest; event-timeframe monitors tick daily rather than per-minute.

4. **REQ-024** — volume-profile computation from 1-minute bars (pure), plus OBI/GEX adapters when a quote and options source is approved.
5. **REQ-091** — two-leg atomic order construction and a cointegration validation harness; the correlation gate's return-series plumbing (this release) is the foundation.

Each promotion ships as: library edit (status + executable mapping + version bump) → new config hash → the §14 evidence attached to the release notes → deployment. No runtime promotion endpoint exists, by design.

11 API reference — new tRPC endpoints

Endpoint	Type	Returns / effect
`engine.strategies`	query	All 100 registry rows: id, name, version, status, direction, timeframe, asset class, first-release flag, eligibility, executable config id, config hash.
`engine.strategyStats`	query	Roll-up: total, approved, eligible, backtest-required, draft, disabled, first-release counts.
`engine.strategyDetail({id})`	query	The complete Universal Strategy Object for one REQ id (all §2 fields).
`engine.strategyDisable({id})`	mutation	Operator kill — demotes to DISABLED for new entries; open positions unaffected. The only runtime status mutation.
`engine.strategyEnable({id})`	mutation	Restores the library status (never a promotion beyond versioned code).
`engine.universe`	query	Engine status (seeds, tracked feeds, market hours, filter constants) + caller's preference + caller's last cycle report.
`engine.universeScan()`	mutation	Forces a full cycle now: tracks seeds (first use), refreshes all feeds, runs filter→rank→assign→monitor for the caller.
`engine.getAutoUniverse` / `engine.setAutoUniverse({enabled, maxMonitors?})`	query / mutation	Per-user opt-in and concurrent-monitor cap (1–20, default 5). Fail-closed reads.

All endpoints are authenticated (`authedQuery`). Unauthenticated calls return 401 — a 404 on any of these paths means the deployment is stale.

12 Operator UI — UniversePanel

Mounted on the Autonomous page directly below EnginePanel. Two surfaces in one card:

12.1 Auto-universe surface

- **Enable/pause button** and **Run universe scan now** (forces a cycle outside the 60s heartbeat).
- **Market-hours badge** — the engine stands down visibly when the market is closed.
- **Last-cycle report** — tracked/seed counts, filtered count, monitors started, and the ranked candidate table: symbol, last, session dollar volume, rvol, assigned strategy, deterministic decision (MONITOR WAIT REJECT) and the exact reason string.

12.2 Registry surface

- **Status chips** — eligible / approved / backtest-required / draft / disabled / first-release counts, live from the registry.
- **Registry table** — first-release and eligible rows by default, expandable to all 100; per-row status badge, direction, timeframe, config hash, and the operator-kill / restore control.
- Eligible rows are tinted — the operator can see at a glance exactly what the bot is allowed to trade.

> **REFETCH CADENCE**
>
> Universe state every 15s; registry every 30s. All data comes from the endpoints in §11 — no client-side computation of trading state.

13 Verification & test plan

13.1 Determinism assertions (the spec's core promise)

1. **Same bars → same range → same decision.** Feed `orb_breakout` a fixed bar series twice; evidence objects must be identical.
2. **Eligibility gate.** `resolveExecutable("REQ-002")` must throw `STRATEGY_NOT_ELIGIBLE`; `resolveExecutable("REQ-001")` must return `req_001_orb_long` with a stable hash across boots.
3. **No runtime promotion.** `strategyEnable` after `strategyDisable` restores the library status and nothing more; a DRAFT strategy cannot become eligible at runtime by any endpoint.
4. **Filter honesty.** A symbol with a stale feed (> 180s) is skipped with a recorded reason; no candidate is ever fabricated from missing data.
5. **Heat gate.** With open risk at 9.6% of equity and a new proposal risking 0.5%, the proposal must REJECT with the projected-heat reason.
6. **Correlation gate.** Candidate feed vs open-position feed with engineered r = 0.9 → REJECT; with the position's feed absent → REJECT (never-estimate).
7. **Heartbeat.** During market hours, a monitor's `timerCount` advances without any UI interaction; outside market hours it is frozen.
8. **Cap discipline.** With `autoUniverseMax = 5` and 5 active monitors, a sixth assigned candidate resolves to WAIT "monitor cap reached".
9. **OCO inheritance.** Auto-started monitors propose at most one ticket per symbol per day (state-machine rule, unchanged).
10. **Trailing isolation.** Disabling REQ-001 mid-position leaves the open position's trailing lifecycle untouched; the trailing engine fires exits normally.

13.2 Smoke checklist (post-deploy, 5 minutes)

Check	Expected
`GET /`	200
`GET /api/trpc/engine.strategyStats` (signed out)	401, not 404
Boot log	`[db] schema ensured` → `Server running`
Autonomous page	UniversePanel renders; registry shows 100 rows, 2 eligible
Enable auto-universe → Run scan	Cycle report appears; every candidate row carries a decision + reason
`DESCRIBE users`	`autoUniverse`, `autoUniverseMax` present with defaults 0 / 5

13.3 Known gaps (declared, not hidden)

- Preflight inputs in `startRun` remain stubbed constants — tracked from the v2.2 audit, unchanged by this release.
- Liquidity score / VPIN declared but not numerically enforced (data adapters pending) — see §8.3.
- Fire-lock persistence, hash-chained audit ledger, dead man's switch, EOD three-way reconciliation remain on the institutional roadmap.
- Backtest-based promotion evidence (§14 chain) intentionally deferred per product decision; the registry's BACKTEST_REQUIRED statuses encode that honestly.

A Appendix A · strategies/types.ts + registry.ts (full source)

A.1 api/engine/strategies/types.ts

```
/**
 * UNIVERSAL STRATEGY OBJECT — Strategy Specification v1.0 §2.
 *
 * Every strategy in the Requi deterministic engine is expressed as ONE
 * schema. The object is DATA: it declares what the strategy needs, how it
 * scans, how it enters, how it is sized, and how it exits — the engine's
 * deterministic pipeline (§3) interprets it. The AI layer may scan, rank
 * and explain; it may never override a parameter, stop, size or gate
 * declared here.
 *
 * Status lifecycle (§14):
 *   DRAFT → BACKTEST_REQUIRED → (BACKTEST_PASS → WALK_FORWARD_PASS →
 *   OUT_OF_SAMPLE_PASS) → PAPER_TRADING → CANARY → APPROVED
 * Only status === "APPROVED" with an executable mapping is eligible for
 * autonomous trading. Any failure demotes back to BACKTEST_REQUIRED.
 * Runtime code can only DEMOTE (APPROVED → DISABLED). Promotion requires
 * validation evidence and a versioned code release — never a silent
 * in-place parameter change (§10/§11).
 */

export type StrategyStatus =
  | "DRAFT"
  | "BACKTEST_REQUIRED"
  | "PAPER_TRADING"
  | "CANARY"
  | "APPROVED"
  | "DISABLED";

export type StrategyDirection = "LONG" | "SHORT";
export type StrategyTimeframe = "INTRADAY" | "SWING" | "EVENT" | "POSITION";
export type AssetClass = "EQUITY" | "ETF" | "OPTIONS" | "FUTURES";
export type Regime = "TRENDING" | "MEAN_REVERTING" | "HIGH_VOL" | "LOW_VOL" | "ANY";

/** Which engine primitives can execute this strategy today. */
export interface ExecutableMapping {
  /** StrategyConfig id in api/engine/config.ts (governance-clamped at load) */
  configId: string;
  /** triggers used by the mapping (must exist in TRIGGERS) */
  triggers: string[];
  /** what is deliberately approximated vs. the full spec (auditable) */
  notes: string;
}

export interface UniverseRules {
  minPrice: number;          // §13 SCAN: price ≥ $5
  minDollarVolume: number;   // dollars per day
  maxSpreadPct: number;      // tight spread requirement
  session: "RTH" | "ETH" | "ANY";
}

export interface UniversalStrategy {
  strategy_id: string;            // "REQ-001"
  name: string;
  version: string;                // semver — trades record the version used (§11)
```

```
/**
 * UNIVERSAL STRATEGY OBJECT — Strategy Specification v1.0 §2.
 *
 * Every strategy in the Requi deterministic engine is expressed as ONE
 * schema. The object is DATA: it declares what the strategy needs, how it
 * scans, how it enters, how it is sized, and how it exits — the engine's
 * deterministic pipeline (§3) interprets it. The AI layer may scan, rank
 * and explain; it may never override a parameter, stop, size or gate
 * declared here.
 *
 * Status lifecycle (§14):
 *   DRAFT → BACKTEST_REQUIRED → (BACKTEST_PASS → WALK_FORWARD_PASS →
 *   OUT_OF_SAMPLE_PASS) → PAPER_TRADING → CANARY → APPROVED
 * Only status === "APPROVED" with an executable mapping is eligible for
 * autonomous trading. Any failure demotes back to BACKTEST_REQUIRED.
 * Runtime code can only DEMOTE (APPROVED → DISABLED). Promotion requires
 * validation evidence and a versioned code release — never a silent
 * in-place parameter change (§10/§11).
 */

export type StrategyStatus =
  | "DRAFT"
  | "BACKTEST_REQUIRED"
  | "PAPER_TRADING"
  | "CANARY"
  | "APPROVED"
  | "DISABLED";

export type StrategyDirection = "LONG" | "SHORT";
export type StrategyTimeframe = "INTRADAY" | "SWING" | "EVENT" | "POSITION";
export type AssetClass = "EQUITY" | "ETF" | "OPTIONS" | "FUTURES";
export type Regime = "TRENDING" | "MEAN_REVERTING" | "HIGH_VOL" | "LOW_VOL" | "ANY";

/** Which engine primitives can execute this strategy today. */
export interface ExecutableMapping {
  /** StrategyConfig id in api/engine/config.ts (governance-clamped at load) */
  configId: string;
  /** triggers used by the mapping (must exist in TRIGGERS) */
  triggers: string[];
  /** what is deliberately approximated vs. the full spec (auditable) */
  notes: string;
}

export interface UniverseRules {
  minPrice: number;          // §13 SCAN: price ≥ $5
  minDollarVolume: number;   // dollars per day
  maxSpreadPct: number;      // tight spread requirement
  session: "RTH" | "ETH" | "ANY";
}

export interface UniversalStrategy {
  strategy_id: string;            // "REQ-001"
  name: string;
  version: string;                // semver — trades record the version used (§11)
  status: StrategyStatus;
  asset_class: AssetClass;
  direction: StrategyDirection;
  timeframe: StrategyTimeframe;
```

```
  universe: UniverseRules;
    regime: { allowed: Regime[] };
    required_data: string[];        // e.g. ["1m_bars"] | ["daily_bars"] | ["options_chain"] |
  ["pairs_second_leg"]
    indicators: string[];

    scan_rules: string[];           // auditable text of each gate
    signal_rules: string[];
    confirmation_rules: string[];

    entry: string;
    position_sizing: { risk_pct: number; hard_cap_pct: number };
    initial_stop: string;
    profit_targets: { t1_r: number; t2_r: number } | null;
    /** How the position hands off to the trailing engine (module: trailing.ts owns every protective exit)
  */
    trailing_stop: string;
    time_exit: string | null;

    liquidity_rules: { min_score: number; vpin_max: number | null };
    portfolio_rules: { heat_max_pct: number; max_pair_correlation: number };
    event_rules: string[];
    no_trade_rules: string[];

    monitor_frequency: string;
    exit_rules: string[];           // priority order per §7 — protective/trailing exits always last
    validation: string[];           // what must be proven before promotion

    /** null = metadata-only (not activatable). Present + APPROVED = eligible. */
    executable: ExecutableMapping | null;
  }

  /** Global deterministic decisions the engine may return (§1). */
  export type EngineDecision =
    | "EXECUTE"
    | "WAIT"
    | "REJECT"
    | "REDUCE_SIZE"
    | "EXIT"
    | "HALT";
```

A.2 api/engine/strategies/registry.ts

```
import { createHash } from "crypto";
import { STRATEGY_LIBRARY, FIRST_RELEASE_IDS } from "./library";
import type { StrategyStatus, UniversalStrategy } from "./types";

/**
 * STRATEGY REGISTRY — the engine's view over the embedded library.
 *
 * Determinism contract (§1/§10/§11):
 *  - The library file is the versioned source of truth. Statuses embedded
 *    in code ship through the normal release pipeline; a config hash over
 *    each strategy object makes any change detectable in the audit trail.
 *  - At runtime the registry can only DEMOTE (APPROVED → DISABLED) — the
 *    operator kill. Promotion requires the §14 evidence chain and a
 *    versioned code release. There is no runtime "make this tradable"
 *    path, by design.
 *  - Eligibility for autonomous trading = status APPROVED + executable
 *    mapping present. Both conditions, always.
 */

/** Runtime demotions (operator kill). Never a promotion channel. */
const runtimeDisabled = new Set<string>();

export function getStrategy(id: string): UniversalStrategy | undefined {
  return STRATEGY_LIBRARY.find((s) => s.strategy_id === id.toUpperCase());
}

export function effectiveStatus(s: UniversalStrategy): StrategyStatus {
  return runtimeDisabled.has(s.strategy_id) ? "DISABLED" : s.status;
}

/** sha256 over the strategy object — recorded with every use (§10). Library
 * objects are built by one code path, so key order is stable; any edit to
 * the library (any level) changes the hash. */
export function configHash(s: UniversalStrategy): string {
  return createHash("sha256").update(JSON.stringify(s)).digest("hex").slice(0, 16);
}

export interface RegistryRow {
  strategyId: string;
  name: string;
  version: string;
  status: StrategyStatus;
  direction: string;
  timeframe: string;
  assetClass: string;
  firstRelease: boolean;
  eligible: boolean;
  executableConfigId: string | null;
  hash: string;
}

export function listRegistry(): RegistryRow[] {
  return STRATEGY_LIBRARY.map((s) => {
    const status = effectiveStatus(s);
```

```
import { createHash } from "crypto";
import { STRATEGY_LIBRARY, FIRST_RELEASE_IDS } from "./library";
import type { StrategyStatus, UniversalStrategy } from "./types";

/**
 * STRATEGY REGISTRY — the engine's view over the embedded library.
 *
 * Determinism contract (§1/§10/§11):
 *  - The library file is the versioned source of truth. Statuses embedded
 *    in code ship through the normal release pipeline; a config hash over
 *    each strategy object makes any change detectable in the audit trail.
 *  - At runtime the registry can only DEMOTE (APPROVED → DISABLED) — the
 *    operator kill. Promotion requires the §14 evidence chain and a
 *    versioned code release. There is no runtime "make this tradable"
 *    path, by design.
 *  - Eligibility for autonomous trading = status APPROVED + executable
 *    mapping present. Both conditions, always.
 */

/** Runtime demotions (operator kill). Never a promotion channel. */
const runtimeDisabled = new Set<string>();

export function getStrategy(id: string): UniversalStrategy | undefined {
  return STRATEGY_LIBRARY.find((s) => s.strategy_id === id.toUpperCase());
}

export function effectiveStatus(s: UniversalStrategy): StrategyStatus {
  return runtimeDisabled.has(s.strategy_id) ? "DISABLED" : s.status;
}

/** sha256 over the strategy object — recorded with every use (§10). Library
 * objects are built by one code path, so key order is stable; any edit to
 * the library (any level) changes the hash. */
export function configHash(s: UniversalStrategy): string {
  return createHash("sha256").update(JSON.stringify(s)).digest("hex").slice(0, 16);
}

export interface RegistryRow {
  strategyId: string;
  name: string;
  version: string;
  status: StrategyStatus;
  direction: string;
  timeframe: string;
  assetClass: string;
  firstRelease: boolean;
  eligible: boolean;
  executableConfigId: string | null;
  hash: string;
}

export function listRegistry(): RegistryRow[] {
  return STRATEGY_LIBRARY.map((s) => {
    const status = effectiveStatus(s);
    return {
      strategyId: s.strategy_id,
      name: s.name,
```

```
status,
      direction: s.direction,
      timeframe: s.timeframe,
      assetClass: s.asset_class,
      firstRelease: (FIRST_RELEASE_IDS as readonly string[]).includes(s.strategy_id),
      eligible: status === "APPROVED" && s.executable !== null,
      executableConfigId: s.executable?.configId ?? null,
      hash: configHash(s),
    };
  });
}

export function registryStats() {
  const rows = listRegistry();
  return {
    total: rows.length,
    approved: rows.filter((r) => r.status === "APPROVED").length,
    eligible: rows.filter((r) => r.eligible).length,
    backtestRequired: rows.filter((r) => r.status === "BACKTEST_REQUIRED").length,
    draft: rows.filter((r) => r.status === "DRAFT").length,
    disabled: rows.filter((r) => r.status === "DISABLED").length,
    firstRelease: rows.filter((r) => r.firstRelease).length,
  };
}

/**
 * Resolve a registry strategy to an executable engine config id.
 * Throws STRATEGY_NOT_ELIGIBLE unless APPROVED + executable — the gate the
 * universe builder and every monitor start must pass (§2).
 */
export function resolveExecutable(id: string): { configId: string; hash: string; version: string } {
  const s = getStrategy(id);
  if (!s) throw new Error(`UNKNOWN_STRATEGY: ${id}`);
  const status = effectiveStatus(s);
  if (status !== "APPROVED" || !s.executable) {
    throw new Error(`STRATEGY_NOT_ELIGIBLE: ${id} is ${status}${s.executable ? "" : " (no executable
mapping)"} — only APPROVED strategies with an executable mapping may trade autonomously (§2)`);
  }
  return { configId: s.executable.configId, hash: configHash(s), version: s.version };
}

/** Eligible (APPROVED + executable) strategies — the autonomous universe's menu. */
export function eligibleStrategies(): RegistryRow[] {
  return listRegistry().filter((r) => r.eligible);
}

/**
 * Operator kill: demote a strategy to DISABLED at runtime. This is the ONLY
 * runtime status mutation the engine permits. It is audited by the caller
 * (engine-router records it in the activity stream).
 */
export function disableStrategy(id: string): { ok: boolean; detail: string } {
  const s = getStrategy(id);
  if (!s) return { ok: false, detail: `unknown strategy ${id}` };
  runtimeDisabled.add(s.strategy_id);
  return { ok: true, detail: `${s.strategy_id} DISABLED by operator — effective immediately for new
entries (open positions unaffected; exits stay with the trailing engine)` };
}
```

```
export function enableStrategy(id: string): { ok: boolean; detail: string } {
  const s = getStrategy(id);
  if (!s) return { ok: false, detail: `unknown strategy ${id}` };
  runtimeDisabled.delete(s.strategy_id);
  const status = effectiveStatus(s);
  return { ok: true, detail: `${s.strategy_id} restored to library status ${status}` };
}
```

B Appendix B · strategies/library.ts (full source)

All 100 Universal Strategy Objects. The first-release ten carry full specification detail; the remaining ninety are compact-but-complete records with their spec rules, data requirements, and the §4 global defaults applied by the builder. Any edit to this file changes the affected strategies' config hashes.

```
import type { AssetClass, Regime, StrategyDirection, StrategyStatus, StrategyTimeframe, UniversalStrategy
} from "./types";

/**
 * STRATEGY LIBRARY — all 100 strategies of Strategy Specification v1.0
 * embedded as Universal Strategy Objects (§2). This file is the versioned
 * source of truth: every change is a code release with a version bump and
 * a new config hash (§10/§11) — the runtime never mutates parameters.
 *
 * First DevOps release (§13): REQ-001, REQ-002, REQ-003, REQ-004, REQ-024,
 * REQ-034, REQ-036, REQ-064, REQ-065, REQ-091.
 *
 * Activation honesty: only strategies whose executable mapping runs on
 * CURRENT engine primitives (SetupMachine + trigger library + ticket gate
 * + trailing engine) are status APPROVED. Everything else is DRAFT or
 * BACKTEST_REQUIRED with the exact blocker recorded in `validation` —
 * promotion requires the §14 evidence chain, never a status edit alone.
 */

export const FIRST_RELEASE_IDS = [
  "REQ-001", "REQ-002", "REQ-003", "REQ-004", "REQ-024",
  "REQ-034", "REQ-036", "REQ-064", "REQ-065", "REQ-091",
] as const;

/* ---------- §4 global risk defaults every strategy inherits ---------- */

const GLOBALS = {
  position_sizing: { risk_pct: 0.5, hard_cap_pct: 1.0 },
  liquidity_rules: { min_score: 0.5, vpin_max: 0.7 as number | null },
  portfolio_rules: { heat_max_pct: 10, max_pair_correlation: 0.7 },
};

const DEFAULT_UNIVERSE = { minPrice: 5, minDollarVolume: 20_000_000, maxSpreadPct: 0.1, session: "RTH" as
const };

interface Def {
  name: string;
  status?: StrategyStatus;
  asset?: AssetClass;
  dir: StrategyDirection;
  tf: StrategyTimeframe;
  regime?: Regime[];
  data: string[];
  indicators?: string[];
  scan?: string[];
  signal: string[];
  confirm?: string[];
  entry: string;
  stop: string;
  targets?: { t1_r: number; t2_r: number } | null;
  trail?: string;
  timeExit?: string | null;
  noTrade?: string[];
  monitor?: string;
  exits?: string[];
  validation?: string[];
  executable?: UniversalStrategy["executable"];
```

```
function def(id: string, d: Def): UniversalStrategy {
  return {
    strategy_id: id,
    name: d.name,
    version: "1.0.0",
    status: d.status ?? "DRAFT",
    asset_class: d.asset ?? "EQUITY",
    direction: d.dir,
    timeframe: d.tf,
    universe: { ...DEFAULT_UNIVERSE },
    regime: { allowed: d.regime ?? ["ANY"] },
    required_data: d.data,
    indicators: d.indicators ?? [],
    scan_rules: d.scan ?? ["Price >= $5", "Dollar volume >= threshold", "Spread <= max", "Regular trading
session"],
    signal_rules: d.signal,
    confirmation_rules: d.confirm ?? [],
    entry: d.entry,
    position_sizing: { ...GLOBALS.position_sizing },
    initial_stop: d.stop,
    profit_targets: d.targets === undefined ? { t1_r: 1.5, t2_r: 2.5 } : d.targets,
    trailing_stop: d.trail ?? "After T1: hand to trailing engine (tightening-only ATR trail, module
trailing.ts owns the exit)",
    time_exit: d.timeExit === undefined ? "Close remaining intraday position by session cutoff" :
d.timeExit,
    liquidity_rules: { ...GLOBALS.liquidity_rules },
    portfolio_rules: { ...GLOBALS.portfolio_rules },
    event_rules: ["Feed down -> HALT new entries", "Missing/stale data -> REJECT (never estimate with an
LLM)"],
    no_trade_rules: d.noTrade ?? ["Low liquidity", "Excessive spread", "Unconfirmed signal", "Market
regime conflict", "Extended entry"],
    monitor_frequency: d.monitor ?? "1-minute bars, every engine tick",
    exit_rules: d.exits ?? ["Kill switch", "Broker fault", "Max-loss", "Thesis invalidation", "Regime
flip", "Protective stop", "Time stop", "Target", "Trailing exit (last)"],
    validation: d.validation ?? ["BACKTEST_PASS", "WALK_FORWARD_PASS", "OUT_OF_SAMPLE_PASS",
"PAPER_TRADING soak", "CANARY"],
    executable: d.executable ?? null,
  };
}

/* ---------- FIRST RELEASE — full specification detail (§13) ---------- */

const FIRST_RELEASE: UniversalStrategy[] = [
  def("REQ-001", {
    name: "Opening Range Breakout Long",
    status: "APPROVED",
    dir: "LONG",
    tf: "INTRADAY",
    regime: ["TRENDING", "ANY"],
    data: ["1m_bars"],
    indicators: ["opening_range", "relative_volume", "atr"],
    scan: ["Price >= $5", "20D median dollar volume >= configured threshold", "Spread <= configured
maximum", "Regular trading session"],
    signal: ["Price > OpeningRangeHigh"],
    confirm: ["RelativeVolume >= 1.5", "Close > OpeningRangeHigh", "MarketAlignment = BULLISH"],
    entry: "Buy-stop above confirmed opening-range breakout",
    stop: "MIN(OpeningRangeLow, StructureStop, ATRStop)",
    targets: { t1_r: 1.5, t2_r: 2.5 },
```

```
timeExit: "Close remaining intraday position by session cutoff",
    executable: {
      configId: "req_001_orb_long",
      triggers: ["orb_breakout", "volume_climax"],
      notes: "Opening range = first 15 RTH minutes computed from session bars; rvol evidence from
volume_climax signal; market alignment approximated by price-vs-VWAP at arm time.",
    },
  }),
  def("REQ-002", {
    name: "Opening Range Breakdown Short",
    status: "BACKTEST_REQUIRED",
    dir: "SHORT",
    tf: "INTRADAY",
    regime: ["TRENDING", "ANY"],
    data: ["1m_bars"],
    indicators: ["opening_range", "relative_volume", "atr"],
    signal: ["Price < OpeningRangeLow"],
    confirm: ["RelativeVolume >= 1.5", "Close < OpeningRangeLow", "MarketAlignment = BEARISH"],
    entry: "Sell-stop below confirmed opening-range breakdown",
    stop: "Above opening-range structure",
    validation: ["Requires short-selling execution path in tickets/portfolio layer", "Standard §14
evidence chain"],
  }),
  def("REQ-003", {
    name: "VWAP Reclaim Long",
    status: "APPROVED",
    dir: "LONG",
    tf: "INTRADAY",
    regime: ["ANY"],
    data: ["1m_bars"],
    indicators: ["vwap", "swing_low", "volume"],
    signal: ["PriorPrice < VWAP AND CrossUp(Price, VWAP)"],
    confirm: ["HigherLow = TRUE", "BarsAboveVWAP >= 2", "VolumeConfirmation = TRUE"],
    entry: "First confirmed hold/retest above VWAP",
    stop: "Below reclaim swing low",
    trail: "After T1: trail beneath confirmed higher lows OR ATR trail (trailing engine)",
    executable: {
      configId: "req_003_vwap_reclaim",
      triggers: ["vwap_reclaim", "volume_climax"],
      notes: "Reclaim+hold enforced by vwap_reclaim trigger (full-reset on close back below); swing-low
stop resolved by SetupMachine; volume confirmation via volume_climax signal.",
    },
  }),
  def("REQ-004", {
    name: "VWAP Rejection Short",
    status: "BACKTEST_REQUIRED",
    dir: "SHORT",
    tf: "INTRADAY",
    regime: ["ANY"],
    data: ["1m_bars"],
    indicators: ["vwap", "relative_strength"],
    signal: ["Price tests VWAP AND Close < VWAP"],
    confirm: ["LowerHigh = TRUE", "WeakRelativeStrength = TRUE"],
    entry: "Sell on confirmed VWAP rejection",
    stop: "Above rejection swing high",
    validation: ["Requires short-selling execution path", "Standard §14 evidence chain"],
  }),
  def("REQ-024", {
```

```
status: "BACKTEST_REQUIRED",
    dir: "LONG",
    tf: "INTRADAY",
    regime: ["TRENDING"],
    data: ["1m_bars", "order_book", "options_gex"],
    indicators: ["volume_profile", "vah", "poc", "obi", "gex"],
    signal: ["Price > VAH + 0.5σ", "Volume >= 1.5 × AverageVolume20", "LiquidityScore > 0.60", "OBI >
+0.40", "GEX regime supports breakout"],
    entry: "Buy on accepted VAH breakout",
    stop: "Below POC",
    targets: null,
    trail: "Target = Entry + 2 × ValueAreaWidth; trail via trailing engine after T1",
    validation: ["Volume-profile computable from 1m bars; OBI + GEX feeds not yet wired — data adapters
required", "Standard §14 evidence chain"],
  }),
  def("REQ-034", {
    name: "20-Day Donchian Breakout Long",
    status: "BACKTEST_REQUIRED",
    dir: "LONG",
    tf: "SWING",
    regime: ["TRENDING"],
    data: ["daily_bars"],
    indicators: ["donchian_20", "atr"],
    signal: ["Price > MAX(High[t-20:t-1])"],
    entry: "Buy-stop above 20-day channel high",
    stop: "10-day opposite channel or ATR-derived stop",
    timeExit: null,
    monitor: "daily close + intraday breakout watch",
    validation: ["Needs multi-session channel context in SetupMachine (20 sessions of daily levels)",
"Standard §14 evidence chain"],
  }),
  def("REQ-036", {
    name: "Turtle Trend Following Long",
    status: "BACKTEST_REQUIRED",
    dir: "LONG",
    tf: "POSITION",
    regime: ["TRENDING"],
    data: ["daily_bars"],
    indicators: ["donchian_20", "donchian_10", "atr20"],
    signal: ["Price > 20-Day High"],
    entry: "Buy-stop above 20-day high; regime must be trending",
    stop: "Exit when Price < 10-Day Low",
    targets: null,
    trail: "Channel-exit managed by trailing engine swing mode",
    timeExit: null,
    monitor: "daily",
    validation: ["Multi-session daily context + position-timeframe holding support required", "Standard
§14 evidence chain"],
  }),
  def("REQ-064", {
    name: "PEAD Long",
    status: "BACKTEST_REQUIRED",
    dir: "LONG",
    tf: "EVENT",
    regime: ["ANY"],
    data: ["earnings_calendar", "estimates", "daily_bars"],
    indicators: ["sue", "announcement_reaction"],
    signal: ["SUE >= configured positive threshold", "Announcement-day reaction confirms"],
```

```
stop: "Below announcement-day low / ATR stop",
    timeExit: "Maximum horizon 30-60 days",
    monitor: "daily",
    validation: ["Earnings calendar + SUE data adapter required", "Standard §14 evidence chain"],
  }),
  def("REQ-065", {
    name: "PEAD Short",
    status: "BACKTEST_REQUIRED",
    dir: "SHORT",
    tf: "EVENT",
    regime: ["ANY"],
    data: ["earnings_calendar", "estimates", "daily_bars"],
    indicators: ["sue", "announcement_reaction"],
    signal: ["SUE <= negative threshold", "Announcement-day downside confirms"],
    entry: "Sell post-announcement downside drift",
    stop: "Above announcement-day high",
    timeExit: "Maximum horizon 30-60 days",
    monitor: "daily",
    validation: ["Short-selling execution path + earnings data adapter required", "Standard §14 evidence
chain"],
  }),
  def("REQ-091", {
    name: "Pairs Mean Reversion",
    status: "BACKTEST_REQUIRED",
    dir: "LONG",
    tf: "SWING",
    regime: ["MEAN_REVERTING"],
    data: ["pairs_second_leg", "daily_bars"],
    indicators: ["spread_z", "cointegration"],
    signal: ["abs(SpreadZ) >= 2 (statistically validated relationship required)"],
    entry: "Enter both legs at SpreadZ extreme",
    stop: "abs(SpreadZ) >= 3 OR StructuralBreak = TRUE",
    targets: null,
    trail: "Target SpreadZ between -0.5 and +0.5",
    timeExit: null,
    monitor: "hourly/daily",
    validation: ["Two-leg simultaneous execution + cointegration validation harness required", "Standard
§14 evidence chain"],
  }),
];

/* ---------- Compact builder for the remaining 90 ---------- */

function c(
  id: string, name: string, dir: StrategyDirection, tf: StrategyTimeframe,
  signal: string[], opts: Partial<Def> = {},
): UniversalStrategy {
  return def(id, { name, dir, tf, data: opts.data ?? ["1m_bars"], signal, entry: opts.entry ?? `${dir ===
"LONG" ? "Buy" : "Sell"} on confirmed signal per spec`, stop: opts.stop ?? "Structure/ATR stop per spec",
...opts });
}

const REMAINING: UniversalStrategy[] = [
  c("REQ-005", "Gap-and-Go Long", "LONG", "INTRADAY", ["Gap up >= threshold with catalyst", "Open holds
above VWAP", "rvol expansion"], { confirm: ["First 15m hold", "MarketAlignment = BULLISH"] }),
  c("REQ-006", "Gap-and-Go Short", "SHORT", "INTRADAY", ["Inverse REQ-005: negative catalyst and downside
confirmation"]),
  c("REQ-007", "Gap Fade Long", "LONG", "INTRADAY", ["Exhaustion gap down into support", "Reversal
```

```
c("REQ-008", "Gap Fade Short", "SHORT", "INTRADAY", ["Inverse REQ-007"]),
  c("REQ-009", "First Pullback Long", "LONG", "INTRADAY", ["Strong opening drive", "First orderly
pullback holds VWAP/ORH"]),
  c("REQ-010", "First Bounce Short", "SHORT", "INTRADAY", ["Inverse REQ-009"]),
  c("REQ-011", "High-of-Day Breakout", "LONG", "INTRADAY", ["Close > session high with volume expansion",
"chase <= limit"]),
  c("REQ-012", "Low-of-Day Breakdown", "SHORT", "INTRADAY", ["Inverse REQ-011"]),
  c("REQ-013", "Premarket High Breakout", "LONG", "INTRADAY", ["Close > premarket high", "RTH
acceptance"], { data: ["1m_bars", "premarket_bars"] }),
  c("REQ-014", "Premarket Low Breakdown", "SHORT", "INTRADAY", ["Inverse REQ-013"], { data: ["1m_bars",
"premarket_bars"] }),
  c("REQ-015", "Volume Expansion Breakout", "LONG", "INTRADAY", ["Volume >= N× 20-bar average at range
break", "follow-through close"]),
  c("REQ-016", "Relative Strength Leader Long", "LONG", "INTRADAY", ["Symbol outperforming benchmark over
window", "trend alignment"], { data: ["1m_bars", "benchmark_bars"] }),
  c("REQ-017", "Relative Weakness Leader Short", "SHORT", "INTRADAY", ["Inverse REQ-016"], { data:
["1m_bars", "benchmark_bars"] }),
  c("REQ-018", "Sector Leader Rotation", "LONG", "SWING", ["Sector relative-strength rotation into
leader"], { data: ["daily_bars", "sector_bars"] }),
  c("REQ-019", "Sector Laggard Short Rotation", "SHORT", "SWING", ["Sector weakness rotation into
laggard"], { data: ["daily_bars", "sector_bars"] }),
  c("REQ-020", "Intraday VWAP Mean Reversion Long", "LONG", "INTRADAY", ["Regime = MEAN_REVERTING",
"Standardized extension below VWAP >= threshold"], { regime: ["MEAN_REVERTING"] }),
  c("REQ-021", "Intraday VWAP Mean Reversion Short", "SHORT", "INTRADAY", ["Inverse REQ-020"], { regime:
["MEAN_REVERTING"] }),
  c("REQ-022", "Anchored VWAP Reversion Long", "LONG", "SWING", ["Extension below event-anchored VWAP",
"reversion trigger"], { data: ["1m_bars", "anchor_events"] }),
  c("REQ-023", "Anchored VWAP Reversion Short", "SHORT", "SWING", ["Inverse REQ-022"], { data:
["1m_bars", "anchor_events"] }),
  c("REQ-025", "Volume Profile VAL Breakdown", "SHORT", "INTRADAY", ["Inverse REQ-024"], { data:
["1m_bars", "order_book", "options_gex"] }),
  c("REQ-026", "POC Mean Reversion", "LONG", "INTRADAY", ["Regime = MEAN_REVERTING", "abs(Price - POC)
standardized >= threshold"], { regime: ["MEAN_REVERTING"], data: ["1m_bars"], targets: null, trail:
"Target = POC" }),
  c("REQ-027", "Low-Volume-Node Momentum", "LONG", "INTRADAY", ["Acceptance into LVN in direction of
breakout"], { data: ["1m_bars"] }),
  c("REQ-028", "Liquidity Sweep Long", "LONG", "INTRADAY", ["Sweep of equal lows / stop pool", "reclaim
confirmation"]),
  c("REQ-029", "Liquidity Sweep Short", "SHORT", "INTRADAY", ["Inverse REQ-028"]),
  c("REQ-030", "Fair Value Gap Long", "LONG", "INTRADAY", ["Bullish FVG fill-and-hold"]),
  c("REQ-031", "Fair Value Gap Short", "SHORT", "INTRADAY", ["Inverse REQ-030"]),
  c("REQ-032", "Order Book Imbalance Long", "LONG", "INTRADAY", ["OBI > threshold sustained", "price
acceptance"], { data: ["1m_bars", "order_book"] }),
  c("REQ-033", "Order Book Imbalance Short", "SHORT", "INTRADAY", ["Inverse REQ-032"], { data:
["1m_bars", "order_book"] }),
  c("REQ-035", "20-Day Donchian Breakdown Short", "SHORT", "SWING", ["Inverse REQ-034"], { data:
["daily_bars"] }),
  c("REQ-037", "Turtle Trend Following Short", "SHORT", "POSITION", ["Inverse REQ-036"], { data:
["daily_bars"] }),
  c("REQ-038", "50-Day Moving-Average Pullback", "LONG", "SWING", ["Uptrend pullback to 50DMA holds",
"reversal confirmation"], { data: ["daily_bars"] }),
  c("REQ-039", "200-Day Trend Recovery", "LONG", "POSITION", ["Reclaim of 200DMA after basing", "trend
confirmation"], { data: ["daily_bars"] }),
  c("REQ-040", "KAMA Adaptive Trend Long", "LONG", "SWING", ["Price > KAMA", "KAMA slope positive"], {
data: ["daily_bars"] }),
  c("REQ-041", "KAMA Adaptive Trend Short", "SHORT", "SWING", ["Inverse REQ-040"], { data: ["daily_bars"]
}),
  c("REQ-042", "VIDYA Adaptive Trend Long", "LONG", "SWING", ["Price > VIDYA", "volatility-adjusted trend
```

```
c("REQ-043", "VIDYA Adaptive Trend Short", "SHORT", "SWING", ["Inverse REQ-042"], { data: ["daily_bars"]
}),
  c("REQ-044", "Modern RSI Oversold Reversion", "LONG", "SWING", ["RSI regime-adaptive oversold",
"reversion trigger"], { data: ["daily_bars"], regime: ["MEAN_REVERTING"] }),
  c("REQ-045", "Modern RSI Overbought Reversion", "SHORT", "SWING", ["Inverse REQ-044"], { data:
["daily_bars"], regime: ["MEAN_REVERTING"] }),
  c("REQ-046", "Modern MACD Trend Long", "LONG", "SWING", ["MACD signal cross with regime filter"], {
data: ["daily_bars"] }),
  c("REQ-047", "Modern MACD Trend Short", "SHORT", "SWING", ["Inverse REQ-046"], { data: ["daily_bars"]
}),
  c("REQ-048", "VWAP/GARCH Bollinger Reversion Long", "LONG", "INTRADAY", ["GARCH-scaled band touch",
"reversion confirmation"], { regime: ["MEAN_REVERTING"] }),
  c("REQ-049", "VWAP/GARCH Bollinger Reversion Short", "SHORT", "INTRADAY", ["Inverse REQ-048"], {
regime: ["MEAN_REVERTING"] }),
  c("REQ-050", "ATR-Normalized Stochastic Long", "LONG", "SWING", ["ATR-normalized stochastic oversold
cross"], { data: ["daily_bars"] }),
  c("REQ-051", "ATR-Normalized Stochastic Short", "SHORT", "SWING", ["Inverse REQ-050"], { data:
["daily_bars"] }),
  c("REQ-052", "Momentum Top-Decile Long", "LONG", "POSITION", ["Universe momentum decile ranking", "top-
decile entry with trend filter"], { data: ["daily_bars", "universe_history"] }),
  c("REQ-053", "Momentum Breakdown Short", "SHORT", "SWING", ["Momentum failure breakdown"], { data:
["daily_bars"] }),
  c("REQ-054", "Jegadeesh-Titman Momentum", "LONG", "POSITION", ["3-12 month formation momentum",
"monthly rebalance"], { data: ["daily_bars", "universe_history"], monitor: "monthly" }),
  c("REQ-055", "Quality + Momentum", "LONG", "POSITION", ["Quality factor screen + momentum overlay"], {
data: ["fundamentals", "daily_bars"] }),
  c("REQ-056", "Value + Momentum", "LONG", "POSITION", ["Value factor screen + momentum overlay"], {
data: ["fundamentals", "daily_bars"] }),
  c("REQ-057", "Quality + Value + Momentum", "LONG", "POSITION", ["Three-factor composite screen"], {
data: ["fundamentals", "daily_bars"] }),
  c("REQ-058", "Low-Volatility Factor", "LONG", "POSITION", ["Low-vol anomaly ranking", "defensive regime
tilt"], { data: ["daily_bars", "universe_history"] }),
  c("REQ-059", "Multi-Factor Alpha", "LONG", "POSITION", ["Composite multi-factor score >= threshold"], {
data: ["fundamentals", "daily_bars", "universe_history"] }),
  c("REQ-060", "Margin-of-Safety Value", "LONG", "POSITION", ["Price < intrinsic estimate by margin-of-
safety threshold"], { data: ["fundamentals", "daily_bars"] }),
  c("REQ-061", "Free-Cash-Flow Inflection", "LONG", "POSITION", ["FCF inflection positive", "valuation
support"], { data: ["fundamentals", "daily_bars"] }),
  c("REQ-062", "Revenue Acceleration", "LONG", "POSITION", ["Sequential revenue acceleration", "estimate
support"], { data: ["fundamentals", "estimates", "daily_bars"] }),
  c("REQ-063", "Margin Expansion", "LONG", "POSITION", ["Gross/operating margin expansion trend"], {
data: ["fundamentals", "daily_bars"] }),
  c("REQ-066", "Earnings Gap Continuation Long", "LONG", "EVENT", ["Positive earnings repricing", "Gap
remains accepted", "Forward revisions positive"], { data: ["earnings_calendar", "estimates", "1m_bars"]
}),
  c("REQ-067", "Earnings Gap Continuation Short", "SHORT", "EVENT", ["Inverse REQ-066"], { data:
["earnings_calendar", "estimates", "1m_bars"] }),
  c("REQ-068", "Earnings Gap Fade Long", "LONG", "EVENT", ["Overextended down-gap post-earnings", "fade
confirmation"], { data: ["earnings_calendar", "1m_bars"] }),
  c("REQ-069", "Earnings Gap Fade Short", "SHORT", "EVENT", ["Inverse REQ-068"], { data:
["earnings_calendar", "1m_bars"] }),
  c("REQ-070", "Estimate Revision Momentum Long", "LONG", "SWING", ["Positive revision breadth/magnitude
trend"], { data: ["estimates", "daily_bars"] }),
  c("REQ-071", "Estimate Revision Short", "SHORT", "SWING", ["Inverse REQ-070"], { data: ["estimates",
"daily_bars"] }),
  c("REQ-072", "Guidance Raise Long", "LONG", "EVENT", ["Guidance raise event", "acceptance
confirmation"], { data: ["earnings_calendar", "news", "daily_bars"] }),
  c("REQ-073", "Guidance Cut Short", "SHORT", "EVENT", ["Inverse REQ-072"], { data: ["earnings_calendar",
```

```
c("REQ-074", "Earnings Straddle: Underpriced Move", "LONG", "EVENT", ["Implied move < model expected
move", "long straddle into event"], { asset: "OPTIONS", data: ["options_chain", "earnings_calendar"] }),
  c("REQ-075", "Earnings IV-Crush Defined-Risk", "SHORT", "EVENT", ["Post-event IV crush harvest with
defined-risk structure"], { asset: "OPTIONS", data: ["options_chain", "earnings_calendar"] }),
  c("REQ-076", "Long Put Breakdown", "LONG", "SWING", ["Technical breakdown + put purchase", "negative
catalyst"], { asset: "OPTIONS", data: ["options_chain", "daily_bars"] }),
  c("REQ-077", "Put Debit Spread", "LONG", "SWING", ["Bearish defined-risk debit structure"], { asset:
"OPTIONS", data: ["options_chain", "daily_bars"] }),
  c("REQ-078", "Call Debit Spread", "LONG", "SWING", ["Bullish defined-risk debit structure"], { asset:
"OPTIONS", data: ["options_chain", "daily_bars"] }),
  c("REQ-079", "Bull Put Spread", "SHORT", "SWING", ["Credit structure at support", "IV rank filter"], {
asset: "OPTIONS", data: ["options_chain", "daily_bars"] }),
  c("REQ-080", "Bear Call Spread", "SHORT", "SWING", ["Credit structure at resistance", "IV rank
filter"], { asset: "OPTIONS", data: ["options_chain", "daily_bars"] }),
  c("REQ-081", "Long Straddle", "LONG", "EVENT", ["Volatility expansion expectation", "event catalyst"],
{ asset: "OPTIONS", data: ["options_chain", "earnings_calendar"] }),
  c("REQ-082", "Long Strangle", "LONG", "EVENT", ["Cheaper wing variant of REQ-081"], { asset: "OPTIONS",
data: ["options_chain", "earnings_calendar"] }),
  c("REQ-083", "Calendar Spread", "LONG", "SWING", ["Term-structure dislocation", "theta/vega profile
within limits"], { asset: "OPTIONS", data: ["options_chain"] }),
  c("REQ-084", "Variance Risk Premium Harvest", "SHORT", "POSITION", ["Implied variance > realized
forecast by threshold", "defined-risk harvest"], { asset: "OPTIONS", data: ["options_chain",
"daily_bars"] }),
  c("REQ-085", "Gamma Flip Trend", "LONG", "INTRADAY", ["Price across gamma flip level", "dealer-hedging
regime supports trend"], { data: ["1m_bars", "options_gex"] }),
  c("REQ-086", "Positive-Gamma Mean Reversion", "LONG", "INTRADAY", ["Positive-GEX regime fade
extremes"], { data: ["1m_bars", "options_gex"], regime: ["MEAN_REVERTING"] }),
  c("REQ-087", "Negative-Gamma Momentum", "LONG", "INTRADAY", ["Negative-GEX regime momentum
continuation"], { data: ["1m_bars", "options_gex"], regime: ["TRENDING", "HIGH_VOL"] }),
  c("REQ-088", "0DTE Gamma Pinning", "LONG", "INTRADAY", ["Pin magnet at large 0DTE strike", "decay
harvest structure"], { asset: "OPTIONS", data: ["options_chain", "options_gex"] }),
  c("REQ-089", "0DTE Gamma Acceleration", "LONG", "INTRADAY", ["Break of pin level accelerates via dealer
hedging"], { asset: "OPTIONS", data: ["options_chain", "options_gex"] }),
  c("REQ-090", "ETF/NAV Statistical Arbitrage", "LONG", "INTRADAY", ["ETF price vs NAV dislocation >=
threshold"], { asset: "ETF", data: ["1m_bars", "nav_feed"] }),
  c("REQ-092", "Institutional Accumulation Composite", "LONG", "POSITION", ["Composite accumulation score
>= threshold (13F, dark-pool, insider inputs)"], { data: ["13f_filings", "dark_pool", "daily_bars"] }),
  c("REQ-093", "Insider Purchase Cluster Long", "LONG", "POSITION", ["Cluster of open-market insider
purchases", "confirmation window"], { data: ["insider_filings", "daily_bars"] }),
  c("REQ-094", "Alternative-Data Composite", "LONG", "POSITION", ["Alt-data composite score >=
threshold"], { data: ["alt_data", "daily_bars"] }),
  c("REQ-095", "News Sentiment Momentum", "LONG", "EVENT", ["Sentiment shock with persistence
confirmation"], { data: ["news", "1m_bars"] }),
  c("REQ-096", "Earnings Call NLP Divergence", "LONG", "EVENT", ["Call-tone vs estimate divergence
signal"], { data: ["transcripts", "estimates", "daily_bars"] }),
  c("REQ-097", "SEC Filing Risk Signal", "SHORT", "POSITION", ["Filing risk-factor deterioration
signal"], { data: ["sec_filings", "daily_bars"] }),
  c("REQ-098", "CFTC COT Contrarian", "LONG", "POSITION", ["Positioning extreme contrarian signal"], {
asset: "FUTURES", data: ["cot_reports", "daily_bars"] }),
  c("REQ-099", "Buyback Support", "LONG", "POSITION", ["Active buyback + support confluence"], { data:
["fundamentals", "daily_bars"] }),
  c("REQ-100", "Smart Money Flow Confirmation", "LONG", "SWING", ["Flow composite confirms directional
thesis"], { data: ["order_flow", "daily_bars"] }),
];

/** The complete 100-strategy library, REQ-001 … REQ-100, spec order. */
export const STRATEGY_LIBRARY: UniversalStrategy[] = (() => {
  const all = [...FIRST_RELEASE, ...REMAINING];
```

```
  return all;
  })();
```

C Appendix C · engine/universe.ts (full source)

```
import { marketDataService, isMarketHours } from "../marketdata/service";
import { groupRthSessions } from "../marketdata/indicators";
import { startMonitor, listMonitors } from "./monitor";
import { evaluateAll } from "./monitor";
import { resolveExecutable, eligibleStrategies } from "./strategies/registry";
import { listAutoUniverseUsers } from "./portfolio";

/**
 * AUTO-UNIVERSE ENGINE — Strategy Specification v1.0 §3 stages 1–2
 * (MARKET SCAN → UNIVERSE FILTER) made deterministic and automatic.
 *
 * Replaces manual symbol tracking: the bot builds its own watchlist.
 *
 *   SEED UNIVERSE (liquid-symbol list, code-versioned)
 *     → track + refresh feeds (shared global feed map)
 *     → UNIVERSE FILTER: price ≥ $5 · session dollar volume ≥ threshold ·
 *       activity floor · RTH session · fresh data (§5: stale → REJECT)
 *     → RANK: rvol × liquidity score
 *     → TOP N per user (default 5)
 *     → STRATEGY ASSIGNMENT: deterministic tape match against APPROVED
 *       registry strategies only (resolveExecutable is the §2 gate)
 *     → startMonitor(...) → SetupMachine → ticket gate → broker
 *     → fill → TRAILING ENGINE owns every exit (coexistence by contract:
 *       this module never touches a protective stop)
 *
 * Deterministic decisions only: EXECUTE path is the existing monitor →
 * ticket pipeline; anything else is WAIT / REJECT / HALT with a recorded
 * reason. The engine never invents missing data (§5).
 *
 * This module also owns the ENGINE HEARTBEAT: ensureUniverseLoop() ticks
 * every 60s in market hours — refresh feeds → evaluate every active
 * monitor (evaluateAll) → run auto-universe cycles. Before this module,
 * monitors only evaluated on manual router calls.
 */

/* ---------- seed universe: broad, liquid, code-versioned ---------- */

const SEED_UNIVERSE: string[] = [
  // Index / sector ETFs
  "SPY", "QQQ", "IWM", "DIA", "XLF", "XLK", "XLE", "XLV", "XLI", "XLP", "XLY", "XLU", "XLC", "XLB",
"XLRE", "SMH", "SOXX", "ARKK", "GLD", "SLV", "USO", "TLT",
  // Mega-cap / high-liquidity single names
  "AAPL", "MSFT", "NVDA", "AMZN", "GOOGL", "META", "TSLA", "AVGO", "AMD", "NFLX", "CRM", "ORCL", "ADBE",
"INTC", "MU", "QCOM", "TXN", "PLTR", "UBER", "SHOP", "SQ", "COIN", "JPM", "BAC", "WFC", "GS", "MS",
"XOM", "CVX", "LLY", "UNH", "JNJ", "PFE", "MRNA", "WMT", "COST", "HD", "NKE", "MCD", "SBUX", "DIS", "BA",
"CAT", "GE", "F", "GM", "RIVN", "SOFI", "HOOD", "SNOW", "DDOG", "NET", "CRWD",
];

/* ---------- universe filter constants (REQ-001 SCAN + §4) ---------- */

const MIN_PRICE = 5;
const MIN_SESSION_DOLLAR_VOLUME = 20_000_000;
const MIN_RVOL = 1.2;                // activity floor for candidates
const STALE_FEED_SECONDS = 180;      // §5: data older than this → REJECT
```

```
import { marketDataService, isMarketHours } from "../marketdata/service";
import { groupRthSessions } from "../marketdata/indicators";
import { startMonitor, listMonitors } from "./monitor";
import { evaluateAll } from "./monitor";
import { resolveExecutable, eligibleStrategies } from "./strategies/registry";
import { listAutoUniverseUsers } from "./portfolio";

/**
 * AUTO-UNIVERSE ENGINE — Strategy Specification v1.0 §3 stages 1–2
 * (MARKET SCAN → UNIVERSE FILTER) made deterministic and automatic.
 *
 * Replaces manual symbol tracking: the bot builds its own watchlist.
 *
 *   SEED UNIVERSE (liquid-symbol list, code-versioned)
 *     → track + refresh feeds (shared global feed map)
 *     → UNIVERSE FILTER: price ≥ $5 · session dollar volume ≥ threshold ·
 *       activity floor · RTH session · fresh data (§5: stale → REJECT)
 *     → RANK: rvol × liquidity score
 *     → TOP N per user (default 5)
 *     → STRATEGY ASSIGNMENT: deterministic tape match against APPROVED
 *       registry strategies only (resolveExecutable is the §2 gate)
 *     → startMonitor(...) → SetupMachine → ticket gate → broker
 *     → fill → TRAILING ENGINE owns every exit (coexistence by contract:
 *       this module never touches a protective stop)
 *
 * Deterministic decisions only: EXECUTE path is the existing monitor →
 * ticket pipeline; anything else is WAIT / REJECT / HALT with a recorded
 * reason. The engine never invents missing data (§5).
 *
 * This module also owns the ENGINE HEARTBEAT: ensureUniverseLoop() ticks
 * every 60s in market hours — refresh feeds → evaluate every active
 * monitor (evaluateAll) → run auto-universe cycles. Before this module,
 * monitors only evaluated on manual router calls.
 */

/* ---------- seed universe: broad, liquid, code-versioned ---------- */

const SEED_UNIVERSE: string[] = [
  // Index / sector ETFs
  "SPY", "QQQ", "IWM", "DIA", "XLF", "XLK", "XLE", "XLV", "XLI", "XLP", "XLY", "XLU", "XLC", "XLB",
"XLRE", "SMH", "SOXX", "ARKK", "GLD", "SLV", "USO", "TLT",
  // Mega-cap / high-liquidity single names
  "AAPL", "MSFT", "NVDA", "AMZN", "GOOGL", "META", "TSLA", "AVGO", "AMD", "NFLX", "CRM", "ORCL", "ADBE",
"INTC", "MU", "QCOM", "TXN", "PLTR", "UBER", "SHOP", "SQ", "COIN", "JPM", "BAC", "WFC", "GS", "MS",
"XOM", "CVX", "LLY", "UNH", "JNJ", "PFE", "MRNA", "WMT", "COST", "HD", "NKE", "MCD", "SBUX", "DIS", "BA",
"CAT", "GE", "F", "GM", "RIVN", "SOFI", "HOOD", "SNOW", "DDOG", "NET", "CRWD",
];

/* ---------- universe filter constants (REQ-001 SCAN + §4) ---------- */

const MIN_PRICE = 5;
const MIN_SESSION_DOLLAR_VOLUME = 20_000_000;
const MIN_RVOL = 1.2;                // activity floor for candidates
const STALE_FEED_SECONDS = 180;      // §5: data older than this → REJECT
const TRACK_BATCH = 5;               // gateway-friendly tracking pace
const TRACK_BATCH_DELAY_MS = 300;
```

```
symbol: string;
  last: number;
  sessionDollarVolume: number;
  rvol: number | null;
  gapPct: number | null;
  vsVwapPct: number | null;
  score: number;
  assignedStrategy: string | null;   // registry id, e.g. "REQ-001"
  assignedConfigId: string | null;
  decision: "MONITOR" | "WAIT" | "REJECT";
  reason: string;
}

export interface UniverseCycleReport {
  userId: number;
  ranAt: number;
  seeds: number;
  tracked: number;
  filtered: number;
  candidates: UniverseCandidate[];
  monitorsStarted: string[];
  maxMonitors: number;
  errors: string[];
}

const lastCycles = new Map<number, UniverseCycleReport>();
const autoStarted = new Set<string>(); // `${userId}:${symbol}` monitors this module started
let seedsTracked = false;

/** Ensure every seed symbol has a tracked feed (batched, gateway-friendly). */
export async function ensureSeedUniverse(): Promise<number> {
  if (seedsTracked) return SEED_UNIVERSE.length;
  for (let i = 0; i < SEED_UNIVERSE.length; i += TRACK_BATCH) {
    const batch = SEED_UNIVERSE.slice(i, i + TRACK_BATCH);
    await Promise.allSettled(batch.map((s) => marketDataService.track(s)));
    await new Promise((r) => setTimeout(r, TRACK_BATCH_DELAY_MS));
  }
  seedsTracked = true;
  return SEED_UNIVERSE.length;
}

/** Deterministic strategy assignment: match tape shape to an APPROVED strategy. */
function assignStrategy(features: {
  sessionAgeMin: number;
  last: number;
  orh: number | null;
  vwap: number | null;
  wasBelowVwap: boolean;
  rvol: number | null;
}): { strategyId: string; configId: string; reason: string } | null {
  const eligible = new Set(eligibleStrategies().map((r) => r.strategyId));
  // REQ-001: opening-range breakout conditions (range formed, price above ORH, rvol hot)
  if (eligible.has("REQ-001") && features.orh !== null && features.sessionAgeMin >= 16 && features.last >
features.orh && (features.rvol ?? 0) >= 1.5) {
    try {
      const ex = resolveExecutable("REQ-001");
      return { strategyId: "REQ-001", configId: ex.configId, reason: `tape matches ORB: last
${features.last} > ORH ${features.orh}, rvol ${features.rvol}` };
```

```
}
  // REQ-003: VWAP reclaim shape (was below, now above)
  if (eligible.has("REQ-003") && features.vwap !== null && features.wasBelowVwap && features.last >
features.vwap) {
    try {
      const ex = resolveExecutable("REQ-003");
      return { strategyId: "REQ-003", configId: ex.configId, reason: `tape matches VWAP reclaim: last
${features.last} > VWAP ${features.vwap} after trading below` };
    } catch { /* falls through */ }
  }
  return null;
}

/** Build one user's universe cycle: filter → rank → assign → start monitors. */
export async function runUniverseCycle(userId: number, maxMonitors: number): Promise<UniverseCycleReport>
{
  const report: UniverseCycleReport = {
    userId,
    ranAt: Date.now(),
    seeds: SEED_UNIVERSE.length,
    tracked: 0,
    filtered: 0,
    candidates: [],
    monitorsStarted: [],
    maxMonitors,
    errors: [],
  };

  const now = Date.now();
  const current = listMonitors(userId).map((m) => m.symbol);
  const openSlots = Math.max(0, maxMonitors - current.length);

  for (const symbol of SEED_UNIVERSE) {
    const feed = marketDataService.get(symbol);
    if (!feed) continue;
    report.tracked++;

    // §5 data-failure rules: stale or errored feeds are REJECTED, never estimated.
    if (feed.error) { report.errors.push(`${symbol}: ${feed.error}`); continue; }
    if (!feed.lastRefresh || now - feed.lastRefresh > STALE_FEED_SECONDS * 1000) continue;
    if (feed.bars.length < 30 || !feed.indicators) continue;

    const sessions = groupRthSessions(feed.bars);
    const days = [...sessions.keys()].sort();
    const today = sessions.get(days[days.length - 1]) ?? [];
    if (today.length < 2) continue;

    const ind = feed.indicators;
    const last = ind.last ?? today[today.length - 1].c;
    const sessionDollarVol = today.reduce((a, b) => a + b.v * ((b.h + b.l + b.c) / 3), 0);

    // UNIVERSE FILTER (REQ-001 SCAN)
    if (last < MIN_PRICE) continue;
    if (sessionDollarVol < MIN_SESSION_DOLLAR_VOLUME) continue;

    // RVOL vs prior session same-bar-count window
    const prior = days.length > 1 ? sessions.get(days[days.length - 2])! : null;
    const priorWindow = prior ? prior.slice(0, today.length) : [];
```

```
  const todayVol = today.reduce((a, b) => a + b.v, 0);
      const rvol = priorVol > 0 ? +(todayVol / priorVol).toFixed(2) : null;
      if (rvol !== null && rvol < MIN_RVOL) continue;

      report.filtered++;

      const gapPct = ind.priorDay && ind.priorDay.close > 0
        ? +(((today[0].o - ind.priorDay.close) / ind.priorDay.close) * 100).toFixed(2)
        : null;
      const vsVwapPct = ind.vwap ? +(((last - ind.vwap) / ind.vwap) * 100).toFixed(2) : null;

      // Opening range (first 15 RTH minutes) for REQ-001 matching
      const range = today.slice(0, 15);
      const orh = range.length >= 15 ? Math.max(...range.map((b) => b.h)) : null;
      const sessionAgeMin = today.length;
      const wasBelowVwap = ind.vwap !== null && today.some((b, i) => {
        // rolling vwap comparison approximated by cumulative-to-bar vwap
        const slice = today.slice(0, i + 1);
        let pv = 0, vol = 0;
        for (const x of slice) { pv += ((x.h + x.l + x.c) / 3) * x.v; vol += x.v; }
        return vol > 0 && b.c < pv / vol;
      });

      const assignment = assignStrategy({ sessionAgeMin, last, orh, vwap: ind.vwap ?? null, wasBelowVwap,
  rvol });
      const score = +(((rvol ?? 1) * Math.log10(sessionDollarVol + 1)).toFixed(2));

      report.candidates.push({
        symbol,
        last,
        sessionDollarVolume: Math.round(sessionDollarVol),
        rvol,
        gapPct,
        vsVwapPct,
        score,
        assignedStrategy: assignment?.strategyId ?? null,
        assignedConfigId: assignment?.configId ?? null,
        decision: assignment ? "MONITOR" : "WAIT",
        reason: assignment?.reason ?? "no APPROVED strategy pattern matches this tape — deterministic
  WAIT",
      });
    }

    // RANK: score desc; only candidates with an assignment and open slots get monitors.
    report.candidates.sort((a, b) => b.score - a.score);
    let slots = openSlots;
    for (const cand of report.candidates) {
      if (slots <= 0) { if (cand.decision === "MONITOR") { cand.decision = "WAIT"; cand.reason = `monitor
  cap reached (${maxMonitors} concurrent) — ranked out`; } continue; }
      if (cand.decision !== "MONITOR" || !cand.assignedConfigId) continue;
      if (current.includes(cand.symbol)) { cand.decision = "WAIT"; cand.reason = "already monitored (manual
  or auto) — no duplicate"; continue; }
      try {
        const started = await startMonitor(userId, cand.symbol, cand.assignedConfigId);
        if (started.ok) {
          autoStarted.add(`${userId}:${cand.symbol}`);
          report.monitorsStarted.push(cand.symbol);
          slots--;
```

```
cand.decision = "REJECT";
        cand.reason = started.detail;
      }
    } catch (e) {
      cand.decision = "REJECT";
      cand.reason = (e as Error).message;
    }
  }

  report.candidates = report.candidates.slice(0, 25); // top of book for the UI
  lastCycles.set(userId, report);
  return report;
}

export function lastUniverseCycle(userId: number): UniverseCycleReport | null {
  return lastCycles.get(userId) ?? null;
}

export function universeStatus() {
  return {
    seeds: SEED_UNIVERSE.length,
    seedsTracked,
    trackedFeeds: marketDataService.list().length,
    marketHours: isMarketHours(),
    filter: { minPrice: MIN_PRICE, minSessionDollarVolume: MIN_SESSION_DOLLAR_VOLUME, minRvol: MIN_RVOL,
staleFeedSeconds: STALE_FEED_SECONDS },
  };
}

/* ---------- engine heartbeat ---------- */

let loop: ReturnType<typeof setInterval> | null = null;
let ticking = false;

/**
 * The engine heartbeat (60s, market hours only):
 *   1. refresh every tracked feed
 *   2. evaluate every active monitor (the tick that drives state machines)
 *   3. run auto-universe cycles for opted-in users
 */
export function ensureUniverseLoop(): void {
  if (loop) return;
  loop = setInterval(() => {
    if (!isMarketHours() || ticking) return;
    ticking = true;
    void (async () => {
      try {
        await marketDataService.refreshAll();
        await evaluateAll();
        const users = await listAutoUniverseUsers();
        if (users.length > 0) await ensureSeedUniverse();
        for (const u of users) {
          await runUniverseCycle(u.id, u.autoUniverseMax).catch(() => undefined);
        }
      } finally {
        ticking = false;
      }
    })();
```

```
  if (typeof loop.unref === "function") loop.unref();
  }
```

D Appendix D · Diffs to existing modules

D.1 triggers.ts — new orb_breakout trigger (REQ-001)

```
/**
 * OPENING RANGE BREAKOUT (REQ-001) — opening range = high/low of the first
 * `range_minutes` RTH bars. Arms when the latest close is above the range
 * high with relative-volume evidence and price within the chase limit.
 * Deterministic: same bars → same range → same decision, live or backtest.
 */
const orbBreakout: TriggerFn = (ctx, p) => {
  const rangeMinutes = Number(p.range_minutes ?? 15);
  const rvolMin = Number(p.rvol_min ?? 1.5);
  const chaseLimitPct = Number(p.chase_limit_pct ?? 3.0);
  if (ctx.session.length < rangeMinutes + 1) {
    return { armed: false, evidence: { reason: "opening_range_forming", have: ctx.session.length, need:
rangeMinutes + 1 } };
  }
  const range = ctx.session.slice(0, rangeMinutes);
  const orh = Math.max(...range.map((b) => b.h));
  const orl = Math.min(...range.map((b) => b.l));
  const last = ctx.session[ctx.session.length - 1].c;
  // Relative volume: range-window volume per bar vs the session average per bar
  const rangeVolPerBar = range.reduce((a, b) => a + b.v, 0) / rangeMinutes;
  const sessionVolPerBar = ctx.session.reduce((a, b) => a + b.v, 0) / ctx.session.length;
  const rvol = sessionVolPerBar > 0 ? rangeVolPerBar / sessionVolPerBar : 0;
  const chasePct = orh > 0 ? ((last - orh) / orh) * 100 : Infinity;
  const armed = last > orh && rvol >= rvolMin && chasePct <= chaseLimitPct;
  return {
    armed,
    evidence: { opening_range_high: orh, opening_range_low: orl, last, rvol: +rvol.toFixed(2), rvol_min:
rvolMin, chase_pct: +chasePct.toFixed(2), chase_limit_pct: chaseLimitPct, range_minutes: rangeMinutes },
  };
};
```

D.2 config.ts — executable mappings for REQ-001 / REQ-003

```
  /* ---------- Strategy Specification v1.0 executable mappings ---------- */
  {
    strategy_id: "req_001_orb_long",
    version: "1.0.0",
    label: "REQ-001 Opening Range Breakout Long",
    entry_variants: [
      { id: "A", priority: 1, trigger_type: "orb_breakout", params: { range_minutes: 15, rvol_min: 1.5,
chase_limit_pct: 3.0 }, min_signals_armed: 1, signals: ["volume_climax"], chase_limit_pct: 3.0,
entry_offset: 0.05 },
      { id: "B", priority: 2, trigger_type: "breakout", params: { level: "session_high", chase_limit_pct:
3.0 }, min_signals_armed: 1, signals: ["volume_climax"], chase_limit_pct: 3.0, entry_offset: 0.05 },
    ],
    stop: { type: "swing_low", offset_atr: 0.5 }, // MIN(ORL, structure, ATR) — conservative structure
stop
    targets: [
      { r_multiple: 1.5, sell_fraction: 0.5 }, // spec: T1 = +1.5R
      { r_multiple: 2.5, sell_fraction: 0.5 }, // spec: T2 = +2.5R
    ],
    hold_timer: { minutes: 10, floor: "vwap" },
    sizing: { max_risk_pct: 0.5, max_position_pct: 25 },
    schedule: { no_entries_after: "15:30", flatten_by: "15:55" },
  },
  {
    strategy_id: "req_003_vwap_reclaim",
    version: "1.0.0",
    label: "REQ-003 VWAP Reclaim Long",
    entry_variants: [
      { id: "A", priority: 1, trigger_type: "vwap_reclaim", params: { hold_minutes: 10 },
min_signals_armed: 1, signals: ["volume_climax"], chase_limit_pct: 3.0, entry_offset: 0.05 },
      { id: "B", priority: 2, trigger_type: "pullback_hold", params: { zone_low: "vwap", zone_high:
"session_high", floor: "vwap", hold_minutes: 10 }, min_signals_armed: 1, signals: ["relative_strength"],
chase_limit_pct: 3.0, entry_offset: 0.05 },
    ],
    stop: { type: "swing_low", offset_atr: 0.5 }, // below reclaim swing low
    targets: [
      { r_multiple: 1.5, sell_fraction: 0.5 },
      { r_multiple: 2.5, sell_fraction: 0.5 },
    ],
    hold_timer: { minutes: 10, floor: "vwap" },
    sizing: { max_risk_pct: 0.5, max_position_pct: 25 },
    schedule: { no_entries_after: "15:30", flatten_by: "15:55" },
  },
];

export function getConfig
```

D.3 risk.ts — §4 globals + heat + correlation

```
/* ---------- Strategy Specification v1.0 §4 global risk limits ---------- */

export const GLOBAL_RISK = {
  /** default per-position risk, percent of equity */
  defaultPositionRiskPct: 0.5,
  /** absolute per-position risk cap — never exceeded, even by config */
  hardPositionRiskCapPct: 1.0,
  /** total open risk across all positions (portfolio heat), percent of equity */
  portfolioHeatMaxPct: 10,
  /** max allowed pairwise correlation between a candidate and open positions */
  maxPairwiseCorrelation: 0.7,
  /** minimum liquidity score to trade */
  liquidityScoreMin: 0.5,
  /** maximum VPIN (flow toxicity) to trade */
  vpinMax: 0.7,
  /** drawdown brake: reduce/halve size beyond this account drawdown, percent */
  drawdownBrakePct: 10,
} as const;

/**
 * PORTFOLIO HEAT (§3 gate 10): total open risk dollars as a percent of
 * equity. Open risk = Σ qty × (mark − stop) floored at 0 for each protected
 * position, plus Σ qty × (entry − stop) for positions still at full risk.
 * Callers pass pre-computed per-position open-risk dollars.
 */
export function portfolioHeatPct(openRiskDollars: number[], accountEquity: number): number {
  if (accountEquity <= 0) return 100;
  const total = openRiskDollars.reduce((a, x) => a + Math.max(x, 0), 0);
  return +((total / accountEquity) * 100).toFixed(2);
}

/**
 * Pairwise return correlation between two bar series (aligned by their last
 * N closes). Deterministic; returns null when either series is too short —
 * callers treat null as "cannot verify → reject" per §5 data-failure rules.
 */
export function returnCorrelation(aCloses: number[], bCloses: number[], window = 30): number | null {
  const n = Math.min(aCloses.length, bCloses.length, window + 1);
  if (n < window / 2) return null;
  const ra: number[] = [];
  const rb: number[] = [];
  for (let i = 1; i < n; i++) {
    const pa = aCloses[aCloses.length - n + i - 1], ca = aCloses[aCloses.length - n + i];
    const pb = bCloses[bCloses.length - n + i - 1], cb = bCloses[bCloses.length - n + i];
    if (pa > 0 && pb > 0) { ra.push(ca / pa - 1); rb.push(cb / pb - 1); }
  }
  if (ra.length < 5) return null;
  const ma = ra.reduce((s, x) => s + x, 0) / ra.length;
  const mb = rb.reduce((s, x) => s + x, 0) / rb.length;
  let cov = 0, va = 0, vb = 0;
  for (let i = 0; i < ra.length; i++) {
    cov += (ra[i] - ma) * (rb[i] - mb);
    va += (ra[i] - ma) ** 2;
    vb += (rb[i] - mb) ** 2;
```

```
/* ---------- Strategy Specification v1.0 §4 global risk limits ---------- */

export const GLOBAL_RISK = {
  /** default per-position risk, percent of equity */
  defaultPositionRiskPct: 0.5,
  /** absolute per-position risk cap — never exceeded, even by config */
  hardPositionRiskCapPct: 1.0,
  /** total open risk across all positions (portfolio heat), percent of equity */
  portfolioHeatMaxPct: 10,
  /** max allowed pairwise correlation between a candidate and open positions */
  maxPairwiseCorrelation: 0.7,
  /** minimum liquidity score to trade */
  liquidityScoreMin: 0.5,
  /** maximum VPIN (flow toxicity) to trade */
  vpinMax: 0.7,
  /** drawdown brake: reduce/halve size beyond this account drawdown, percent */
  drawdownBrakePct: 10,
} as const;

/**
 * PORTFOLIO HEAT (§3 gate 10): total open risk dollars as a percent of
 * equity. Open risk = Σ qty × (mark − stop) floored at 0 for each protected
 * position, plus Σ qty × (entry − stop) for positions still at full risk.
 * Callers pass pre-computed per-position open-risk dollars.
 */
export function portfolioHeatPct(openRiskDollars: number[], accountEquity: number): number {
  if (accountEquity <= 0) return 100;
  const total = openRiskDollars.reduce((a, x) => a + Math.max(x, 0), 0);
  return +((total / accountEquity) * 100).toFixed(2);
}

/**
 * Pairwise return correlation between two bar series (aligned by their last
 * N closes). Deterministic; returns null when either series is too short —
 * callers treat null as "cannot verify → reject" per §5 data-failure rules.
 */
export function returnCorrelation(aCloses: number[], bCloses: number[], window = 30): number | null {
  const n = Math.min(aCloses.length, bCloses.length, window + 1);
  if (n < window / 2) return null;
  const ra: number[] = [];
  const rb: number[] = [];
  for (let i = 1; i < n; i++) {
    const pa = aCloses[aCloses.length - n + i - 1], ca = aCloses[aCloses.length - n + i];
    const pb = bCloses[bCloses.length - n + i - 1], cb = bCloses[bCloses.length - n + i];
    if (pa > 0 && pb > 0) { ra.push(ca / pa - 1); rb.push(cb / pb - 1); }
  }
  if (ra.length < 5) return null;
  const ma = ra.reduce((s, x) => s + x, 0) / ra.length;
  const mb = rb.reduce((s, x) => s + x, 0) / rb.length;
  let cov = 0, va = 0, vb = 0;
  for (let i = 0; i < ra.length; i++) {
    cov += (ra[i] - ma) * (rb[i] - mb);
    va += (ra[i] - ma) ** 2;
    vb += (rb[i] - mb) ** 2;
  }
  if (va === 0 || vb === 0) return 0; // a series that never moves cannot co-move
```

D.4 monitor.ts — proposal gates (imports)

```
import { proposeTicket, autoExecuteTicket } from "../queries/tickets";
import { sizePosition, portfolioHeatPct, returnCorrelation, GLOBAL_RISK } from "./risk";
import { isAutoExecuteEnabled, openRiskDollars, listPositions } from "./portfolio";
```

D.5 monitor.ts — heat + correlation gate block

```
      } else if (await (async () => {
        /* §3 gate 10 — PORTFOLIO HEAT: projected open risk must stay ≤ 10% of equity. */
        const heatNow = portfolioHeatPct(await openRiskDollars(userId), PAPER_EQUITY);
        const projected = +((heatNow + (size.dollarRisk / PAPER_EQUITY) * 100)).toFixed(2);
        if (projected > GLOBAL_RISK.portfolioHeatMaxPct) {
          detail = `proposal blocked by PORTFOLIO HEAT gate — projected ${projected}% >
${GLOBAL_RISK.portfolioHeatMaxPct}% cap (current ${heatNow}% + new $${size.dollarRisk})`;
          return true;
        }
        /* §3 gate 11 — CORRELATION: candidate vs every open position; > 0.70 or unverifiable → REJECT
(§5). */
        const open = await listPositions(userId, 50);
        const candCloses = todayBars.slice(-31).map((b) => b.c);
        for (const p of open.filter((x) => x.status === "OPEN" && x.symbol !== sym)) {
          const pFeed = marketDataService.get(p.symbol);
          if (!pFeed || pFeed.bars.length < 10) {
            detail = `proposal blocked by CORRELATION gate — cannot verify vs open position ${p.symbol}
(no feed data; §5 never-estimate rule)`;
            return true;
          }
          const corr = returnCorrelation(candCloses, pFeed.bars.slice(-31).map((b) => b.c));
          if (corr === null) {
            detail = `proposal blocked by CORRELATION gate — insufficient overlapping bars vs ${p.symbol}
(§5 never-estimate rule)`;
            return true;
          }
          if (corr > GLOBAL_RISK.maxPairwiseCorrelation) {
            detail = `proposal blocked by CORRELATION gate — r=${corr} vs open position ${p.symbol}
exceeds ${GLOBAL_RISK.maxPairwiseCorrelation}`;
            return true;
          }
        }
        return false;
      })()) {
        // detail set by the gate above — proposal dies here, deterministic REJECT
      } else {
```

D.6 portfolio.ts — auto-universe queries + open-risk dollars

```
/* ---------- auto-universe preference (Strategy Spec §3 stages 1–2) ---------- */

export async function isAutoUniverseEnabled(userId: number): Promise<{ enabled: boolean; maxMonitors:
number }> {
  try {
    const db = getDb();
    const [u] = await db.select({ autoUniverse: users.autoUniverse, autoUniverseMax:
users.autoUniverseMax }).from(users).where(eq(users.id, userId)).limit(1);
    return { enabled: u?.autoUniverse === true, maxMonitors: u?.autoUniverseMax ?? 5 };
  } catch {
    return { enabled: false, maxMonitors: 5 }; // fail CLOSED — manual tracking only
  }
}

export async function setAutoUniverse(userId: number, enabled: boolean, maxMonitors?: number): Promise<{
enabled: boolean; maxMonitors: number }> {
  const db = getDb();
  const current = await isAutoUniverseEnabled(userId);
  const max = Math.min(Math.max(maxMonitors ?? current.maxMonitors, 1), 20);
  await db.update(users).set({ autoUniverse: enabled, autoUniverseMax: max }).where(eq(users.id,
userId));
  return { enabled, maxMonitors: max };
}

/** Every user who opted in to the auto-universe engine (drives the heartbeat cycles). */
export async function listAutoUniverseUsers(): Promise<Array<{ id: number; autoUniverseMax: number }>> {
  try {
    const db = getDb();
    const rows = await db.select({ id: users.id, autoUniverseMax: users.autoUniverseMax
}).from(users).where(eq(users.autoUniverse, true));
    return rows.map((r) => ({ id: Number(r.id), autoUniverseMax: r.autoUniverseMax ?? 5 }));
  } catch {
    return []; // fail CLOSED — no auto-universe cycles without a readable roster
  }
}

/**
 * Open-risk dollars per open position for the portfolio-heat gate (§3 gate 10).
 * Risk = qty × (mark/entry − protective stop), floored at 0. Positions with
 * no protective stop are counted at FULL notional risk — an unprotected
 * position is the most conservative possible heat contribution.
 */
export async function openRiskDollars(userId: number): Promise<number[]> {
  try {
    const db = getDb();
    const rows = await db.select().from(positions).where(and(eq(positions.userId, userId),
eq(positions.status, "OPEN")));
    return rows.map((p) => {
      const entry = Number(p.avgEntry);
      const stop = p.stopPrice !== null ? Number(p.stopPrice) : null;
      if (stop === null) return p.quantity * entry; // unprotected → full notional at risk
      return Math.max(p.quantity * (entry - stop), 0);
    });
  } catch {
```

```
/* ---------- auto-universe preference (Strategy Spec §3 stages 1–2) ---------- */

export async function isAutoUniverseEnabled(userId: number): Promise<{ enabled: boolean; maxMonitors:
number }> {
  try {
    const db = getDb();
    const [u] = await db.select({ autoUniverse: users.autoUniverse, autoUniverseMax:
users.autoUniverseMax }).from(users).where(eq(users.id, userId)).limit(1);
    return { enabled: u?.autoUniverse === true, maxMonitors: u?.autoUniverseMax ?? 5 };
  } catch {
    return { enabled: false, maxMonitors: 5 }; // fail CLOSED — manual tracking only
  }
}

export async function setAutoUniverse(userId: number, enabled: boolean, maxMonitors?: number): Promise<{
enabled: boolean; maxMonitors: number }> {
  const db = getDb();
  const current = await isAutoUniverseEnabled(userId);
  const max = Math.min(Math.max(maxMonitors ?? current.maxMonitors, 1), 20);
  await db.update(users).set({ autoUniverse: enabled, autoUniverseMax: max }).where(eq(users.id,
userId));
  return { enabled, maxMonitors: max };
}

/** Every user who opted in to the auto-universe engine (drives the heartbeat cycles). */
export async function listAutoUniverseUsers(): Promise<Array<{ id: number; autoUniverseMax: number }>> {
  try {
    const db = getDb();
    const rows = await db.select({ id: users.id, autoUniverseMax: users.autoUniverseMax
}).from(users).where(eq(users.autoUniverse, true));
    return rows.map((r) => ({ id: Number(r.id), autoUniverseMax: r.autoUniverseMax ?? 5 }));
  } catch {
    return []; // fail CLOSED — no auto-universe cycles without a readable roster
  }
}

/**
 * Open-risk dollars per open position for the portfolio-heat gate (§3 gate 10).
 * Risk = qty × (mark/entry − protective stop), floored at 0. Positions with
 * no protective stop are counted at FULL notional risk — an unprotected
 * position is the most conservative possible heat contribution.
 */
export async function openRiskDollars(userId: number): Promise<number[]> {
  try {
    const db = getDb();
    const rows = await db.select().from(positions).where(and(eq(positions.userId, userId),
eq(positions.status, "OPEN")));
    return rows.map((p) => {
      const entry = Number(p.avgEntry);
      const stop = p.stopPrice !== null ? Number(p.stopPrice) : null;
      if (stop === null) return p.quantity * entry; // unprotected → full notional at risk
      return Math.max(p.quantity * (entry - stop), 0);
    });
  } catch {
    return [];
  }
}
```

D.7 schema.ts + ensure-schema.ts — new user columns

```
  /** When true, the auto-universe engine builds this user's watchlist and starts monitors automatically
(Strategy Spec §3 stages 1–2). Default off. */
  autoUniverse: boolean("autoUniverse").notNull().default(false),
  /** Max concurrent auto-universe monitors per user. */
  autoUniverseMax: int("autoUniverseMax").notNull().default(5),
```

```
  // Auto-universe engine (Strategy Spec §3 stages 1–2): bot-built watchlist, default OFF
  `ALTER TABLE users ADD COLUMN autoUniverse TINYINT(1) NOT NULL DEFAULT 0 AFTER stopMode`,
  `ALTER TABLE users ADD COLUMN autoUniverseMax INT NOT NULL DEFAULT 5 AFTER autoUniverse`,
```

D.8 engine-router.ts — new endpoints

```
  /* ---------- strategy registry (Strategy Spec v1.0) ---------- */

  /** All 100 embedded strategies: status, eligibility, config hash (§10). */
  strategies: authedQuery.query(() => listRegistry()),

  /** Registry roll-up: totals by status, eligible count, first-release count. */
  strategyStats: authedQuery.query(() => registryStats()),

  /** Full Universal Strategy Object for one strategy (§2 schema). */
  strategyDetail: authedQuery.input(z.object({ id: z.string().min(1).max(12) })).query(({ input }) => {
    const s = getStrategy(input.id);
    if (!s) throw new Error(`Unknown strategy ${input.id}`);
    return s;
  }),

  /** Operator kill: demote a strategy to DISABLED (the only runtime status mutation). */
  strategyDisable: authedQuery.input(z.object({ id: z.string().min(1).max(12) })).mutation(({ input }) =>
disableStrategy(input.id)),

  /** Restore a disabled strategy to its library status (never a promotion). */
  strategyEnable: authedQuery.input(z.object({ id: z.string().min(1).max(12) })).mutation(({ input }) =>
enableStrategy(input.id)),

  /* ---------- auto-universe engine (§3 stages 1–2) ---------- */

  /** Universe engine status + this user's last cycle report. */
  universe: authedQuery.query(async ({ ctx }) => ({
    ...universeStatus(),
    preference: await isAutoUniverseEnabled(ctx.user.id),
    lastCycle: lastUniverseCycle(ctx.user.id),
  })),

  /** Force an auto-universe cycle right now (tracks seeds on first use). */
  universeScan: authedQuery.mutation(async ({ ctx }) => {
    const pref = await isAutoUniverseEnabled(ctx.user.id);
    await ensureSeedUniverse();
    await marketDataService.refreshAll();
    return runUniverseCycle(ctx.user.id, pref.maxMonitors);
  }),

  /** Auto-universe preference: enabled + max concurrent monitors. */
  getAutoUniverse: authedQuery.query(async ({ ctx }) => isAutoUniverseEnabled(ctx.user.id)),

  setAutoUniverse: authedQuery
    .input(z.object({ enabled: z.boolean(), maxMonitors: z.number().int().min(1).max(20).optional() }))
    .mutation(async ({ ctx, input }) => setAutoUniverse(ctx.user.id, input.enabled, input.maxMonitors)),
```

D.9 boot.ts — heartbeat wiring

```
import { ensureUniverseLoop } from "./engine/universe";
```

```
  ensureUniverseLoop(); // 60s engine heartbeat — feed refresh → monitor evaluation → auto-universe
cycles
```

E Appendix E · UniversePanel.tsx (full source)

```
import { useState } from 'react';
import { Globe2, Library, Play, ShieldBan, ShieldCheck } from 'lucide-react';
import { trpc } from '@/providers/trpc';
import { cn } from '@/lib/utils';

const STATUS_STYLE: Record<string, string> = {
  APPROVED: 'bg-emerald-50 text-emerald-700 ring-emerald-200',
  BACKTEST_REQUIRED: 'bg-amber-50 text-amber-700 ring-amber-200',
  PAPER_TRADING: 'bg-sky-50 text-sky-700 ring-sky-200',
  CANARY: 'bg-violet-50 text-violet-700 ring-violet-200',
  DRAFT: 'bg-slate-100 text-slate-600 ring-slate-200',
  DISABLED: 'bg-rose-50 text-rose-700 ring-rose-200',
};

const DECISION_STYLE: Record<string, string> = {
  MONITOR: 'text-emerald-600',
  WAIT: 'text-amber-600',
  REJECT: 'text-rose-600',
};

/**
 * UNIVERSE PANEL — the deterministic strategy engine surface.
 * Auto-universe (bot-built watchlist, Strategy Spec §3 stages 1–2) and the
 * 100-strategy registry with the §2 APPROVED-only eligibility gate.
 */
export default function UniversePanel() {
  const utils = trpc.useUtils();
  const [showAll, setShowAll] = useState(false);

  const { data: universe } = trpc.engine.universe.useQuery(undefined, { refetchInterval: 15000 });
  const { data: stats } = trpc.engine.strategyStats.useQuery(undefined, { refetchInterval: 30000 });
  const { data: registry } = trpc.engine.strategies.useQuery(undefined, { refetchInterval: 30000 });

  const setAutoUniverseMut = trpc.engine.setAutoUniverse.useMutation({
    onSettled: () => void utils.engine.universe.invalidate(),
  });
  const scanMut = trpc.engine.universeScan.useMutation({
    onSettled: () => {
      void utils.engine.universe.invalidate();
      void utils.engine.monitors.invalidate();
    },
  });
  const disableMut = trpc.engine.strategyDisable.useMutation({ onSettled: () => void
utils.engine.strategies.invalidate() });
  const enableMut = trpc.engine.strategyEnable.useMutation({ onSettled: () => void
utils.engine.strategies.invalidate() });

  const pref = universe?.preference;
  const cycle = universe?.lastCycle;
  const rows = showAll ? registry ?? [] : (registry ?? []).filter((r) => r.firstRelease || r.eligible);

  return (
    <section className="glass rounded-2xl p-6">
      {/* ── auto-universe engine ── */}
```

```
import { useState } from 'react';
import { Globe2, Library, Play, ShieldBan, ShieldCheck } from 'lucide-react';
import { trpc } from '@/providers/trpc';
import { cn } from '@/lib/utils';

const STATUS_STYLE: Record<string, string> = {
  APPROVED: 'bg-emerald-50 text-emerald-700 ring-emerald-200',
  BACKTEST_REQUIRED: 'bg-amber-50 text-amber-700 ring-amber-200',
  PAPER_TRADING: 'bg-sky-50 text-sky-700 ring-sky-200',
  CANARY: 'bg-violet-50 text-violet-700 ring-violet-200',
  DRAFT: 'bg-slate-100 text-slate-600 ring-slate-200',
  DISABLED: 'bg-rose-50 text-rose-700 ring-rose-200',
};

const DECISION_STYLE: Record<string, string> = {
  MONITOR: 'text-emerald-600',
  WAIT: 'text-amber-600',
  REJECT: 'text-rose-600',
};

/**
 * UNIVERSE PANEL — the deterministic strategy engine surface.
 * Auto-universe (bot-built watchlist, Strategy Spec §3 stages 1–2) and the
 * 100-strategy registry with the §2 APPROVED-only eligibility gate.
 */
export default function UniversePanel() {
  const utils = trpc.useUtils();
  const [showAll, setShowAll] = useState(false);

  const { data: universe } = trpc.engine.universe.useQuery(undefined, { refetchInterval: 15000 });
  const { data: stats } = trpc.engine.strategyStats.useQuery(undefined, { refetchInterval: 30000 });
  const { data: registry } = trpc.engine.strategies.useQuery(undefined, { refetchInterval: 30000 });

  const setAutoUniverseMut = trpc.engine.setAutoUniverse.useMutation({
    onSettled: () => void utils.engine.universe.invalidate(),
  });
  const scanMut = trpc.engine.universeScan.useMutation({
    onSettled: () => {
      void utils.engine.universe.invalidate();
      void utils.engine.monitors.invalidate();
    },
  });
  const disableMut = trpc.engine.strategyDisable.useMutation({ onSettled: () => void
utils.engine.strategies.invalidate() });
  const enableMut = trpc.engine.strategyEnable.useMutation({ onSettled: () => void
utils.engine.strategies.invalidate() });

  const pref = universe?.preference;
  const cycle = universe?.lastCycle;
  const rows = showAll ? registry ?? [] : (registry ?? []).filter((r) => r.firstRelease || r.eligible);

  return (
    <section className="glass rounded-2xl p-6">
      {/* ── auto-universe engine ── */}
      <div className="mb-5 flex items-start gap-3">
        <div className="grid h-9 w-9 shrink-0 place-items-center rounded-xl bg-gradient-to-br from-
indigo-500 to-sky-500 text-white">
```

```
</div>
        <div className="flex-1">
          <h3 className="text-sm font-bold text-slate-900">Auto-universe engine — the bot builds the
watchlist</h3>
          <p className="text-xs text-slate-500">
            Seed universe → filter (price ≥ $5 · $20M session dollar volume · rvol ≥ 1.2 · fresh data) →
rank → assign APPROVED strategies → start monitors. Exits always stay with the trailing engine.
          </p>
        </div>
        {universe && (
          <span className={cn('rounded-full px-2.5 py-1 text-[10px] font-bold ring-1',
universe.marketHours ? 'bg-emerald-50 text-emerald-700 ring-emerald-200' : 'bg-slate-100 text-slate-500
ring-slate-200')}>
            {universe.marketHours ? 'MARKET OPEN' : 'MARKET CLOSED'}
          </span>
        )}
      </div>

      <div className="mb-5 flex flex-wrap items-center gap-2">
        <button
          onClick={() => setAutoUniverseMut.mutate({ enabled: !pref?.enabled })}
          disabled={setAutoUniverseMut.isPending || !pref}
          className={cn(
            'h-9 rounded-lg px-4 text-xs font-bold transition',
            pref?.enabled ? 'bg-emerald-600 text-white hover:bg-emerald-700' : 'bg-slate-900 text-white
hover:bg-slate-800',
            'disabled:opacity-50',
          )}
        >
          {pref?.enabled ? 'Auto-universe ON — click to pause' : 'Enable auto-universe'}
        </button>
        <button
          onClick={() => scanMut.mutate()}
          disabled={scanMut.isPending}
          className="flex h-9 items-center gap-1.5 rounded-lg bg-sky-600 px-4 text-xs font-bold text-
white transition hover:bg-sky-700 disabled:opacity-50"
        >
          <Play className="h-3.5 w-3.5" /> {scanMut.isPending ? 'Scanning…' : 'Run universe scan now'}
        </button>
        {universe && (
          <span className="text-[11px] text-slate-500">
            {universe.seeds} seeds · {universe.trackedFeeds} feeds tracked · max {pref?.maxMonitors ?? 5}
concurrent monitors
          </span>
        )}
      </div>

      {/* last cycle candidates */}
      {cycle && (
        <div className="mb-6 overflow-hidden rounded-xl border border-slate-200">
          <div className="border-b border-slate-200 bg-slate-50 px-3 py-2 text-[11px] font-semibold text-
slate-600">
            Last cycle {new Date(cycle.ranAt).toLocaleTimeString()} — {cycle.tracked}/{cycle.seeds}
tracked · {cycle.filtered} passed filter · {cycle.monitorsStarted.length} monitors started
            {cycle.monitorsStarted.length > 0 && <span className="ml-2 text-emerald-600">→
{cycle.monitorsStarted.join(', ')}</span>}
          </div>
          {cycle.candidates.length === 0 ? (
            <p className="px-3 py-3 text-xs text-slate-400">No candidates passed the universe filter this
```

```
cycle.</p>
          ) : (
            <table className="w-full text-left text-[11px]">
              <thead className="bg-white text-slate-400">
                <tr>
                  <th className="px-3 py-1.5 font-semibold">Symbol</th>
                  <th className="px-3 py-1.5 font-semibold">Last</th>
                  <th className="px-3 py-1.5 font-semibold">$ Vol</th>
                  <th className="px-3 py-1.5 font-semibold">RVOL</th>
                  <th className="px-3 py-1.5 font-semibold">Strategy</th>
                  <th className="px-3 py-1.5 font-semibold">Decision</th>
                  <th className="px-3 py-1.5 font-semibold">Reason</th>
                </tr>
              </thead>
              <tbody className="divide-y divide-slate-100 bg-white/60">
                {cycle.candidates.map((c) => (
                  <tr key={c.symbol}>
                    <td className="px-3 py-1.5 font-bold text-slate-800">{c.symbol}</td>
                    <td className="px-3 py-1.5 text-slate-600">${c.last.toFixed(2)}</td>
                    <td className="px-3 py-1.5 text-slate-600">{(c.sessionDollarVolume /
1e6).toFixed(0)}M</td>
                    <td className="px-3 py-1.5 text-slate-600">{c.rvol ?? '—'}</td>
                    <td className="px-3 py-1.5 font-semibold text-slate-700">{c.assignedStrategy ?? '—'}
</td>
                    <td className={cn('px-3 py-1.5 font-bold', DECISION_STYLE[c.decision])}>{c.decision}
</td>
                    <td className="max-w-[280px] truncate px-3 py-1.5 text-slate-500" title={c.reason}>
{c.reason}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
        </div>
      )}

      {/* ── strategy registry ── */}
      <div className="mb-4 flex items-start gap-3">
        <div className="grid h-9 w-9 shrink-0 place-items-center rounded-xl bg-gradient-to-br from-slate-
700 to-slate-900 text-white">
          <Library className="h-4.5 w-4.5" />
        </div>
        <div className="flex-1">
          <h3 className="text-sm font-bold text-slate-900">Strategy registry — 100 strategies embedded
(Spec v1.0)</h3>
          <p className="text-xs text-slate-500">
            Only APPROVED strategies with an executable mapping may trade autonomously (§2). Operator
kill is the only runtime status change; promotion ships through the §14 evidence chain + versioned
release.
          </p>
        </div>
      </div>

      {stats && (
        <div className="mb-4 flex flex-wrap gap-2 text-[10px] font-bold">
          <span className="rounded-full bg-emerald-50 px-2.5 py-1 text-emerald-700 ring-1 ring-emerald-
200">{stats.eligible} ELIGIBLE</span>
          <span className="rounded-full bg-emerald-50/60 px-2.5 py-1 text-emerald-700 ring-1 ring-
```

```
<span className="rounded-full bg-amber-50 px-2.5 py-1 text-amber-700 ring-1 ring-amber-200">
{stats.backtestRequired} BACKTEST_REQUIRED</span>
          <span className="rounded-full bg-slate-100 px-2.5 py-1 text-slate-600 ring-1 ring-slate-200">
{stats.draft} DRAFT</span>
          <span className="rounded-full bg-rose-50 px-2.5 py-1 text-rose-700 ring-1 ring-rose-200">
{stats.disabled} DISABLED</span>
          <span className="rounded-full bg-sky-50 px-2.5 py-1 text-sky-700 ring-1 ring-sky-200">
{stats.firstRelease} FIRST RELEASE</span>
        </div>
      )}

      <div className="overflow-hidden rounded-xl border border-slate-200">
        <table className="w-full text-left text-[11px]">
          <thead className="bg-slate-50 text-slate-400">
            <tr>
              <th className="px-3 py-2 font-semibold">ID</th>
              <th className="px-3 py-2 font-semibold">Strategy</th>
              <th className="px-3 py-2 font-semibold">Dir</th>
              <th className="px-3 py-2 font-semibold">TF</th>
              <th className="px-3 py-2 font-semibold">Status</th>
              <th className="px-3 py-2 font-semibold">Hash</th>
              <th className="px-3 py-2 font-semibold"></th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100 bg-white/60">
            {rows.map((r) => (
              <tr key={r.strategyId} className={cn(r.eligible && 'bg-emerald-50/40')}>
                <td className="px-3 py-1.5 font-mono font-bold text-slate-800">
                  {r.strategyId}
                  {r.firstRelease && <span className="ml-1.5 rounded bg-sky-100 px-1 py-0.5 text-[9px]
font-bold text-sky-700">1ST</span>}
                </td>
                <td className="px-3 py-1.5 text-slate-700">{r.name}</td>
                <td className={cn('px-3 py-1.5 font-bold', r.direction === 'LONG' ? 'text-emerald-600' :
'text-rose-600')}>{r.direction}</td>
                <td className="px-3 py-1.5 text-slate-500">{r.timeframe}</td>
                <td className="px-3 py-1.5">
                  <span className={cn('rounded-full px-2 py-0.5 text-[10px] font-bold ring-1',
STATUS_STYLE[r.status])}>{r.status}</span>
                </td>
                <td className="px-3 py-1.5 font-mono text-[10px] text-slate-400" title="Config hash
(§10)">{r.hash}</td>
                <td className="px-3 py-1.5 text-right">
                  {r.status === 'DISABLED' ? (
                    <button onClick={() => enableMut.mutate({ id: r.strategyId })} className="text-
emerald-600 hover:text-emerald-700" title="Restore to library status">
                      <ShieldCheck className="h-3.5 w-3.5" />
                    </button>
                  ) : (
                    <button onClick={() => disableMut.mutate({ id: r.strategyId })} className="text-
slate-300 hover:text-rose-500" title="Operator kill — DISABLE">
                      <ShieldBan className="h-3.5 w-3.5" />
                    </button>
                  )}
                </td>
              </tr>
            ))}
          </tbody>
```

```
</div>
      <button onClick={() => setShowAll((v) => !v)} className="mt-2 text-[11px] font-semibold text-sky-
600 hover:text-sky-700">
        {showAll ? 'Show first-release + eligible only' : `Show all ${registry?.length ?? 100}
strategies`}
      </button>
    </section>
  );
}
```